

Every Camera

Программа управления камерами Canon (gphoto2), SPTT (CSDU-429), Infra (Tanho SW1300 SWIR), Sentry (Princeton Instruments CCD через демон imagerd_rt), ASI — всенебный имиджер (Princeton Instruments PIXIS через PICAM SDK + фильтровое колесо) и Japan — всенебный имиджер Hamamatsu (DCAM-API + то же самое фильтровое колесо).

Два режима работы: консольный (headless, с живой консолью измерений) и графический (PyQt5).

Отдельные программы для наблюдателя:

- `monitor_app.py` — состояние всех камер в локальной сети плитками (MQTT — вторым источником);
- `viewer_app.py` — просмотрщик кадров: по локальной сети любой снятый кадр, через HiveMQ — последний;
- `focus_app.py` — помощник настройки резкости: живой поток с изменяемыми параметрами (только по локальной сети).

Структура проекта

```
main.py                — точка входа: измерения (консоль / GUI)
monitor_app.py         — точка входа: монитор камер в сети (отдельная программа)
viewer_app.py          — точка входа: просмотрщик кадров (наблюдатель)
focus_app.py          — точка входа: настройка резкости (наблюдатель)
setup_app.py           — мастер настройки конфигурации (GUI + консоль)
update.py              — утилита обновления программы
schedule_generator.py  — генератор расписания по углу Солнца

cameras/
  __init__.py          — пакет драйверов камер
  cannon_driver.py     — Canon DSLR: подключение, настройка, съёмка (gphoto2)
  sptt_driver.py       — CSDU-429: USB, съёмка, FITS
  sptt_load_firmware.py — загрузчик прошивки CSDU-429 (FX2 + FPGA)
  infra_driver.py      — Tanho SW1300 SWIR: USB, съёмка, TIFF/PNG/FITS
  sentry_driver.py     — Princeton CCD via imagerd_rt: супервизор демона
  asi_driver.py        — ASI (всенебный имиджер): рабочий цикл, тёмные кадры, расписание
  japan_driver.py      — Japan (всенебный имиджер Hamamatsu): то же, два режима
  common/
    filterwheel.py     — фильтровое колесо Animatics SmartMotor (COM/USB)
    filterwheel_sim.py — симулятор колеса
    schedule.py        — слоты расписания, тайминг циклов, оценка тёмных кадров
    sun.py             — высота Солнца (astropy)
    timeutil.py        — одно определение UTC для имён, шапок и Солнца
    cfgparse.py        — чтение значений из config.json без исключений
    fits.py            — 18 общих карт шапки FITS и сама запись файла
  asi/
    picam.py           — ctypes-биндинг PICAM SDK (libpicam.so)
    camera.py          — PixisCamera: экспозиция, биннинг, gain, охлаждение, съёмка
    camera_sim.py      — SimCamera: те же методы, синтетические кадры, без SDK
    devices.py         — выбор бэкенда (picam/sim, порт/sim)
    config.py          — секция asi из config.json в виде типизированных объектов
    exposure.py        — авто-экспозиция и разбиение слота по переэкспозиции
    fits.py            — запись FITS: общее ядро плюс шапка imagerd_rt
    paths.py           — дерево архива UTC YYYY/MM/DD и старые имена файлов
    seqno.py           — сквозной счётчик кадров архива (шапка SEQNO)
    filterwheel.py, filterwheel_sim.py, schedule.py, sun.py, timeutil.py
                        — реэкспорт из common/ (совместимость со старыми импортами)
  japan/
    dcamsdk/
      vendor/
        camera.py      — HamamatsuCamera: экспозиция, биннинг, скорость счит., съёмка
        camera_sim.py  — SimCamera: те же методы, синтетические кадры, без SDK
        devices.py     — выбор бэкенда (dcam/sim, порт/sim)
        config.py      — секция japan из config.json в виде типизированных объектов
```

fits.py	– запись FITS: общее ядро со словами этой камеры
paths.py	– плоские имена YYYYmddTHHMMSS_<фильтр>[_bg].fits
console_ui.py	– консоль измерений: живой экран в отдельном потоке + лог
utils.py	– конфигурация, расписание, сеть, системная информация
mqtt_client.py	– MQTT публикация и подписка (консоль + GUI)
worker_common.py	– общая обвязка воркеров: статусы, публикация кадров
frame_archive.py	– чтение архива кадров, конвертация в JPEG, метрика резкости
camera_service.py	– потокобезопасная прослойка между воркером и сетью
frame_server.py	– HTTP-сервер кадров на машине с камерой
discovery.py	– обнаружение камер в локальной сети (UDP)
net_client.py	– HTTP-клиент и Qt-помощники для программ наблюдателя
gui_app.py	– графический интерфейс (вкладки всех камер + Monitor)
monitor.py	– виджет мониторинга (MQTT + локальные файлы статуса)
generate_pdf.py	– генератор PDF документации из README.md
config.json	– конфигурация (не перезаписывается при обновлении)
firmware/	– бинарные файлы прошивки SPTT (fx2, fpga)
infra_lib/	– библиотека libTanhAPI.so для камеры SW1300
tests/	– тесты (pytest, оборудование не требуется)
test_asi_schedule.py	– расписание и тайминг циклов ASI
test_asi_picam.py	– сверка ctypes-биндинга PICAM с заголовком
test_console_ui.py	– консоль измерений
cannon_driver.py	– совместимость (re-export из cameras/)
sptt_driver.py	– совместимость (re-export из cameras/)
infra_driver.py	– совместимость (re-export из cameras/)
sentry_driver.py	– совместимость (re-export из cameras/)
asi_driver.py	– совместимость (re-export из cameras/)
japan_driver.py	– совместимость (re-export из cameras/)
sptt_load_firmware.py	– совместимость (re-export из cameras/)
cannon-camera/	– submodule (оригинальный репозиторий Canon)
SPTT-CAM/	– submodule (оригинальный репозиторий SPTT)
infra-camera/	– submodule (оригинальный репозиторий SW1300)
asi-camera/	– submodule (оригинальная программа измерений ASI)
japan-camera/	– submodule (оригинальная программа измерений Hamamatsu)

Субмодули хранятся только для справки: код every-camera из них ничего не импортирует и работает при отсутствующих папках submodule.

Установка

```
git clone --recurse-submodules <url>
cd every-camera
pip install -r requirements-<роль>.txt
```

Файлы зависимостей по ролям

Ставится файл своей роли — общее ядро (requirements.txt: numpy, Pillow, paho-mqtt) он подключает сам. Узлу с одной камерой зависимости остальных не нужны, и PyQt5 на безголовом узле тоже не нужен.

Файл	Для чего	Что добавляет к ядру
`requirements-cannon.txt`	узел с камерой Canon	`gphoto2-cffi`
`requirements-sptt.txt`	узел с камерой SPTT	`pyusb`, `astropy`
`requirements-infra.txt`	узел с камерой Infra	`opencv-python-headless` (необяз.), `astropy`
`requirements-asi.txt`	узел с камерой ASI	`pyserial`, `astropy`
`requirements-japan.txt`	узел с камерой Japan	`pyserial`, `astropy`; нужен DCAM-API SDK
`requirements-sentry.txt`	узел с камерой Sentry	`astropy`; нужен установленный `imagerd_rt`

`requirements-gui.txt`	плюс GUI (`gui_app.py`, `setup_app.py`)	`PyQt5` + `PyQt5-Qt5` + `PyQt5-sip`
`requirements-observer.txt`	машина наблюдателя	ядро + GUI
`requirements-dev.txt`	разработка	`pytest`, `fpdf2`

Например, узел с камерой ASI и графической оболочкой:

```
pip install -r requirements-asi.txt -r requirements-gui.txt
```

Установка на CentOS 7.8 (Python 3.11.7)

У каждой строки в этих файлах указан диапазон версий, а не просто `>=`. Это не косметика: glibc на CentOS 7 — 2.17, и часть свежих пакетов публикует колёса только под glibc 2.28, а собрать их из исходников системным gcc 4.8 нельзя.

Диапазоны подобраны так, чтобы pip выбирал последнюю версию с колесом manylinux2014_x86_64.

Проверить, что всё поставится **колёсами**, до самой установки:

```
python3.11 -m pip download -r requirements-asi.txt -d /tmp/wheels --only-binary=:all:
```

Если что-то не скачалось — опустить верхнюю границу у этой строки. Если, наоборот, колесом ставится более новая версия, границу можно поднимать; комментарий у строки нужно поправить вместе с ней.

gphoto2-cffi колёс не публикует вовсе и собирается из исходников:

```
sudo yum install gcc libffi-devel libgphoto2-devel
```

ASI дополнительно:

```
# PICAM SDK: установить libpicam.so в /usr/local/lib (или указать путь)
export PICAM_LIB=/opt/PrincetonInstruments/picam/runtime/libpicam.so

# доступ к последовательному порту фильтрового колеса
sudo usermod -aG dialout $USER      # и перелогиниться
```

Без PICAM SDK драйвер всё равно запускается — с `asi.camera.backend = "sim"` (и `asi.filter_wheel.port = "sim"`): работают расписание, тёмные кадры, запись FITS, консоль, LAN-сервер и живой просмотр — всё, кроме реального железа.

Установка на машину наблюдателя (Windows / Linux)

viewer_app.py и focus_app.py не импортируют драйверы камер — ни gphoto2, ни pyusb, ни libTanhAPI.so. На компьютер наблюдателя достаточно скопировать репозиторий и поставить:

```
pip install -r requirements-observer.txt
pip install astropy      # опционально: без него FITS читается встроенным парсером
```

raho-mqtt нужен только для вкладки HiveMQ в просмотрщике; по локальной сети всё работает на стандартной библиотеке.

Быстрый старт

Консольный режим (без дисплея)

```
# Canon DSLR
```

```
python main.py --type cannon

# SPTT (CSDU-429)
python main.py --type sptt

# Infra (SW1300 SWIR)
python main.py --type infra

# Sentry (Princeton CCD via imagerd_rt)
python main.py --type sentry

# ASI (всенебный имиджер: Princeton PIXIS через PICAM)
python main.py --type asi

# Japan (всенебный имиджер Hamamatsu через DCAM-API)
python main.py --type japan

# Обычные строки лога вместо живого экрана
python main.py --type asi --verbose
```

Графический режим

```
# Автоматически (если есть дисплей)
python main.py

# Явно
python main.py --gui

# Только одна камера
python main.py --gui --type cannon
python main.py --gui --type sptt
python main.py --gui --type infra
python main.py --gui --type sentry
python main.py --gui --type asi
```

Мониторинг (отдельная программа)

```
python monitor_app.py      # плитки всех камер, найденных в локальной сети
```

Программы наблюдателя

```
python viewer_app.py      # просмотр кадров: LAN — любой кадр, HiveMQ — последний
python focus_app.py       # настройка резкости: живой поток (только LAN)
```

Мастер настройки (setup_app.py)

setup_app.py — отдельная программа для настройки конфигурации. Читает и записывает тот же config.json, что использует main.py.

GUI-режим

```
# Все камеры (вкладки Canon / SPTT / Infra / Sentry / ASI / Japan / MQTT / LAN Server / General)
python setup_app.py

# Только одна камера
python setup_app.py --type sentry
python setup_app.py --type asi
python setup_app.py --type japan

# Другой файл конфигурации
```

```
python setup_app.py --config /path/to/config.json
```

Окно мастера содержит вкладки для каждой камеры со всеми настройками. Вкладка **Sentry** включает редактор слотов расписания (таблица с кнопками «Add slot», «Remove selected», «Move up/down»). Вкладка **ASI** содержит настройки камеры, охлаждения, фильтрового колеса, координат станции и таблицу слотов расписания — колонки Delta/Binning работают в режиме time, колонка Seconds — в режиме sun. Вкладка **Japan** устроена так же, но короче: у этой камеры нет охлаждения, авто-экспозиции и разбиения слота, а в списке режимов два пункта вместо трёх. Вкладка **LAN Server** настраивает HTTP-сервер кадров, к которому подключаются viewer_app.py и focus_app.py, а вкладка **General** — имя узла (node_name) и директорию статусов.

Кнопки окна: **Save** (сохранить), **Load config...** (загрузить другой файл), **Reset to defaults** (сбросить в заводские значения), **Close**.

Консольный режим

```
# Интерактивная настройка всех камер (выбор в начале)
python setup_app.py --console

# Только одна камера
python setup_app.py --console --type infra

# Другой файл конфигурации
python setup_app.py --console --config /path/to/config.json
```

Консольный мастер задаёт все параметры через серию вопросов в терминале с подсказками и значениями по умолчанию.

> **Примечание.** При первом запуске main.py в консольном режиме, если конфигурация пуста, программа автоматически запустит мастер настройки для выбранной камеры. setup_app.py — это то же самое, но без привязки к режиму измерений.

Конфигурация

Файл config.json в корне проекта. Создаётся автоматически при первом запуске.

```
{
  "cannon": {
    "instance_name": "",
    "output_dir": "/path/to/output",
    "schedule_file": "/path/to/schedule.txt",
    "capture_seconds": [0, 30],
    "camcfg_file": "",
    "shutterspeed": ""
  },
  "sptt": {
    "instance_name": "",
    "output_dir": "/path/to/output",
    "exposure": 0.88,
    "period_us": null,
    "trigmode": null,
    "gain": 100,
    "binning": 0,
    "encoding": 1,
    "target_temp": null,
    "firmware_dir": "",
    "capture_seconds": [0, 30]
  },
  "infra": {
    "instance_name": "",
    "output_dir": "/path/to/output",
```

```

"schedule_file": "/path/to/schedule.txt",
"capture_seconds": [0, 30],
"exposure_us": 1000.0,
"gain": 0,
"roi": "1280x1024",
"save_format": "tiff"
},
"sentry": {
  "instance_name": "",
  "imagerd_rt_dir": "/usr/local/imagerd_rt",
  "output_dir": "/path/to/fits/archive",
  "device_id": "ASI1",
  "site_id": "SITE",
  "zenith_angle_start": -10,
  "schedule_len_ms": 1440000,
  "imaging_mode": "schedule",
  "capture_seconds": [0, 30],
  "slots": [
    {
      "filter_num": 1,
      "start_ms": 0,
      "exposure_ms": 55000,
      "binning": 2,
      "gain": 3,
      "readout_ms": 5000
    }
  ]
},
"asi": {
  "instance_name": "",
  "output_dir": "/data/asi",
  "mode": "sun",
  "sun_max_angle": -10.0,
  "t_start": "20:00",
  "dark_frames": 3,
  "dead_time": 5.0,
  "wait_for_enter": true,
  "schedule_file": "",
  "schedule": [
    { "filter": 1, "exposure": 55, "seconds": [0] },
    { "filter": 2, "exposure": 55, "seconds": [30] }
  ],
  "camera": {
    "backend": "picam",
    "readout_speed": 2.0,
    "binning": 4,
    "gain": 1,
    "frame_timeout_ms": 30000,
    "readout_control_mode": 1,
    "shutter_timing_mode": 1,
    "output_signal": 4,
    "demo": false,
    "demo_model": "Pixis1024F"
  },
  "cooling": {
    "enabled": true,
    "target_temp": -60.0,
    "tolerance": 3.0,
    "wait_on_start": true,
    "wait_timeout": 1800.0,
    "warm_on_exit": true,
    "warm_temp": 13.0,
    "warm_timeout": 900.0
  },
  "filter_wheel": {
    "port": "/dev/ttyUSB0",
    "baudrate": 9600,
    "move_timeout": 8.0
  }
}

```

```

    },
    "location": {
      "lat": 51.810646,
      "lon": 103.075555,
      "elevation": 658
    }
  },
  "japan": {
    "instance_name": "",
    "output_dir": "/data/japan/20260729",
    "mode": "sun",
    "sun_max_angle": -10.0,
    "t_start": "20:00",
    "dark_frames": 3,
    "dead_time": 5.0,
    "wait_for_enter": true,
    "schedule_file": "",
    "schedule": [
      {"filter": 3, "exposure": 30, "seconds": [0, 30]},
      {"filter": 5, "exposure": 60, "seconds": [0]}
    ],
    "camera": {
      "backend": "dcam",
      "readout_speed": 2,
      "binning": 1,
      "frame_timeout_ms": 1000
    },
    "filter_wheel": {
      "port": "/dev/ttyUSB0",
      "baudrate": 9600,
      "move_timeout": 8.0
    },
    "location": {
      "lat": 53.324236,
      "lon": 107.741264,
      "elevation": 515
    }
  },
  "mqtt": {
    "enabled": false,
    "host": "broker.hivemq.com",
    "port": 1883,
    "user": "",
    "password": "",
    "prefix": "every_camera",
    "tls": false
  },
  "server": {
    "enabled": true,
    "bind": "0.0.0.0",
    "port": 8765,
    "port_search": 20,
    "discovery": true,
    "discovery_port": 45455,
    "max_list": 2000,
    "focus_ttl": 60
  },
  "node_name": "Tory-1",
  "status_dir": ""
}

```

Параметры

Canon (`cannon`)

- **instance_name** — имя экземпляра (если пусто — генерируется автоматически: `Cannon\_<IP>`)

- **output_dir** — директория для сохранения JPEG снимков
- **schedule_file** — путь к файлу расписания
- **capture_seconds** — секунды каждой минуты для съёмки (по умолчанию `[0, 30]`)
- **camcfg_file** — путь к .ini файлу настроек камеры (если пусто — `camcfg\_<Модель>.ini` рядом с программой; файл генерируется автоматически при первом запуске, относительный путь отсчитывается от каталога программы)
- **shutterspeed** — выдержка в том виде, как её называет сама камера: `"1/125"`, `"10.3"`, `"30"`. Пусто — выдержка берётся из camcfg_*.ini и не трогается. Заданное значение применяется **поверх** camcfg_*.ini, поэтому единственная настройка, которую обычно меняют, лежит в config.json и переносится вместе с ним (сам camcfg_*.ini в .gitignore). Если камера такой выдержки не поддерживает, при запуске выводится ошибка со списком допустимых значений, а съёмка продолжается со старой выдержкой.

Список выдержек зависит от модели (Canon EOS 6D: от 30 до 1/4000). Камера должна быть в ручном режиме — при autoexposuremode / shootingmode не Manual выводится предупреждение, поскольку в P/Av/Tv камера подбирает выдержку сама.

Выдержку можно менять и на лету — через focus_app.py или MQTT-команду capture_frame; config.json задаёт лишь состояние на старте.

- > **Не путать с `cannon-camera/cannon\_config.json`**. В сабмодуле лежит конфиг старой отдельной программы — плоская схема с ключами mqtt_host, mqtt_port, status_dir
- > и без camcfg_file / shutterspeed / capture_seconds. Эта программа его не читает:
- > все настройки Canon берутся из секции cannon в корневом config.json.

SPTT (`sptt`)

- **instance_name** — имя экземпляра (если пусто — `SPTT\_<IP>`)
- **output_dir** — директория для сохранения FITS файлов
- **exposure** — экспозиция в секундах (по умолчанию `0.88`)
- **period_us** — период кадра в микросекундах. `null` (по умолчанию) — вычисляется автоматически: экспозиция плюс 75 000 мкс на считывание сенсора и передачу по USB, но не меньше аппаратного минимума, который камера сообщает сама
- **trigmode** — режим запуска: `null` или `0` — непрерывный, далее `1` — внешний, 2 программный, 3 bulb, 5 внешний одиночный, 7 bulb одиночный. Вне непрерывного режима период не влияет на накопление и передаётся камере как есть
- **gain** — усиление (0–1023, по умолчанию `100`)
- **binning** — биннинг (`0` = 1×1, `1` = 2×2, `3` = 4×4)
- **encoding** — разрядность оцифровки (`0` = 8 бит, `1` = 12 бит); в кадре значения в любом случае приведены к общей шкале 0–65535, см. [«Шкала интенсивности»](#шкала-интенсивности-065535)
- **target_temp** — целевая температура ПЗС в °C (`null` = охлаждение не задано)
- **firmware_dir** — директория с файлами прошивки `fx2\_firmware.bin` и `fpga\_bitstream.bin`. Если пусто — берётся firmware/ в корне программы (рядом с main.py). Относительный путь отсчитывается от корня программы, абсолютный используется как есть. Если файлов там нет, камера не поднимется, и в журнале будет указан путь, по которому их искали
- **capture_seconds** — секунды каждой минуты для публикации в MQTT

Экспозиция и период кадра

У CSDU-429 экспозиция и период кадра — два независимых регистра. В непрерывном режиме период задаёт интервал кадра, поэтому **период короче экспозиции не замедляет камеру, а обрезает накопление**: кадры получаются тёмными, и ни камера, ни программа об этом не сообщают. Прошивка спокойно принимает обе величины и продолжает возвращать их в блоке состояния.

Поэтому период здесь не задаётся вручную, а выводится из экспозиции и пересчитывается при каждом её изменении — в том числе при живой смене через `focus_app.py`, MQTT или GUI. Заданный вручную `period_us` короче экспозиции поднимается до безопасного значения с предупреждением в журнале; в непрерывном режиме иначе быть не может.

После установки параметров программа читает состояние камеры и сверяет фактическую экспозицию и период с запрошенными. Расхождение — это отказ, а не предупреждение: на старте камера просто не запускается (лучше не начать ночь, чем отснять её на неверной экспозиции), при смене параметров на лету ошибка возвращается в `focus_app` или в MQTT-ответ, а расписание продолжает работать на последнем исправном значении.

В заголовке архивного кадра:

Ключ	Значение
`EXPTIME`	экспозиция, о которой сообщает сама камера — именно её нужно использовать
`EXPREQ`	экспозиция, которая была запрошена; при исправной работе совпадает с `EXPTIME`
`PERIOD`	фактический период кадра в микросекундах
`MINPERD`	минимальный период, допустимый прошивкой при текущем биннинге и ROI
`TRIGMODE`	режим запуска
`FRAMELAG`	сколько секунд прошло от начала накопления до получения кадра

DATE-OBS теперь берётся из момента, когда камере разрешено начать накопление, а не из момента чтения. Плановый снимок в `:00/:30` начинает собственный кадр вместо того, чтобы забирать кадр, пролежавший в FIFO с прошлого снимка.

> Частота кадров в предпросмотре при длинной экспозиции закономерно падает: при 0.88 с
> это примерно один кадр в секунду вместо прежних тринадцати. Прежние тринадцать были
> кадрами по 75 мс независимо от настройки — это следствие исправления, а не регрессия.

Мёртвое время между кадрами

Камера работает свободным ходом, а следующее накопление начинается не само по себе, а в тот момент, когда программа дочитала кадр и переинициализировала FIFO. Поэтому всё, что происходит между «кадр готов» и «FIFO переинициализирован», превращается в мёртвое время — и, поскольку оно зависит от того, куда попал очередной опрос и что ещё шло по шине, при одной и той же экспозиции оно получалось разным.

Убраны два источника этого разброса:

- **Опрос готовности кадра** идёт с шагом 5 мс, как в программе вендора, а не 50 мс. Шаг опроса целиком уходит в мёртвое время, причём каждый раз в разной доле.
- **Опрос состояния камеры** в цикле измерений выполняется на частоте публикации статуса (раз в `STATUS_MIN_INTERVAL`), а не десять раз в секунду. Раньше между кадрами по управляющей конечной точке шло десять служебных обменов в секунду — по той же шине, по которой считывается кадр.

Фактическая задержка записывается в каждый кадр ключом `FRAMELAG` — это секунды между

разрешением на накопление и получением готового кадра. По нему разброс мёртвого времени видно прямо в архиве, без внешних измерений.

Infra (`infra`)

- instance_name** — имя экземпляра (если пусто — `Infra\_<IP>`)
- output_dir** — директория для сохранения снимков
- schedule_file** — путь к файлу расписания
- capture_seconds** — секунды каждой минуты для съёмки (по умолчанию `[0, 30]`)
- exposure_us** — экспозиция в микросекундах (по умолчанию `1000.0`)
- gain** — усиление (0–120, по умолчанию `0`)
- roi** — область захвата (`"1280x1024"` — полный кадр, `"1280x256"` — горизонтальная полоса)
- save_format** — формат сохранения: `"tiff"` (16-bit raw), `"png"` (8-bit нормализованный), `"fits"` (FITS с метаданными)

Камера Tanho THCAMSW1300 (сенсор Sony IMX990-AABA-C), SWIR-диапазон, 12-bit ADC.

Отсчёты приводятся к общей шкале 0–65535 сразу после деинтерливинга, см.

[«Шкала интенсивности»](#шкала-интенсивности-065535).

Sentry (`sentry`)

Камера Princeton Instruments CCD управляется через сторонний демон `imagerd_rt` (Keo Scientific, бинарный файл). Драйвер every-camera выступает **супервизором**: генерирует `schedule.conf`, запускает демон, читает его выходные файлы и публикует статус/кадры через MQTT.

- instance_name** — имя экземпляра (если пусто — `Sentry\_<IP>`)
- imagerd_rt_dir** — директория установки `imagerd_rt` (по умолчанию `/usr/local/imagerd_rt`)
- output_dir** — директория для архива FITS файлов (опционально)
- device_id** — идентификатор устройства камеры (например `"ASI1"`)
- site_id** — идентификатор станции (используется в именах файлов)
- zenith_angle_start** — зенитный угол Солнца, при котором начинаются измерения (градусы; отрицательное = Солнце ниже горизонта, по умолчанию `-10`)
- schedule_len_ms** — длительность одного цикла расписания в миллисекундах (по умолчанию `1440000` = 24 мин)
- imaging_mode** — режим съёмки: `"schedule"` (по слотам расписания) или `"rapid"` (непрерывная съёмка)
- capture_seconds** — секунды каждой минуты для опроса статуса демона / публикации в MQTT
- slots** — список слотов расписания (см. ниже)

Слоты расписания Sentry

Каждый слот описывает один сеанс съёмки с определённым светофильтром внутри цикла. Параметры слота:

Поле	Описание
<code>`filter_num`</code>	Номер светофильтра
<code>`start_ms`</code>	Время начала слота от начала цикла (мс)
<code>`exposure_ms`</code>	Длительность экспозиции (мс)
<code>`binning`</code>	Биннинг: <code>`1`</code> , <code>`2`</code> или <code>`4`</code>
<code>`gain`</code>	Усиление ПЗС (1–3 для ProEM; допустимые значения зависят от модели)
<code>`readout_ms`</code>	Время перемещения колеса фильтров + считывания матрицы (мс)

Пример — два слота, фильтры 1 и 2, каждые 24 секунды:

```
"slots": [  
  { "filter_num": 1, "start_ms": 0,      "exposure_ms": 8000, "binning": 2, "gain": 3, "readout_ms": 3000 },  
  { "filter_num": 2, "start_ms": 12000, "exposure_ms": 8000, "binning": 2, "gain": 3, "readout_ms": 3000 }  
]
```

Из этих слотов драйвер автоматически генерирует файл `schedule.conf` в формате `imagerd_rt` при каждом запуске.

Файловый интерфейс `imagerd_rt`

Файл	Описание
<code><dir>/schedule.conf</code>	Расписание (генерируется драйвером)
<code><dir>/aux/image.fits</code>	Последний захваченный кадр
<code><dir>/aux/meta-data.txt</code>	Метаданные текущего кадра (пары ключ-значение, TAB)
<code><dir>/aux/seqno.txt</code>	Счётчик кадров
<code><dir>/aux/stop_file</code>	Управление: запись <code>"10"</code> = стоп, <code>"11"</code> = возобновить

ASI (`asi`)

Всенебный имиджер: ПЗС-камера **Princeton Instruments PIXIS** через **PICAM SDK** и фильтровое колесо **Animatics SmartMotor** по последовательному порту. В отличие от `sentry`, который лишь управляет готовым демоном `imagerd_rt`, драйвер `asi` работает с оборудованием напрямую.

Основные параметры:

- output_dir** — директория для FITS-файлов
- instance_name** — имя экземпляра (пусто = `ASI_<последний октет IP>`)
- mode** — режим расписания: `"sun"` (по секундам минуты, пока темно), `"time"` (цикл по часам) или `"sun_cycle"` (цикл, запускаемый углом Солнца)
- sun_max_angle** — режимы `"sun"` и `"sun_cycle"`: съёмка идёт, пока высота Солнца $\leq$ этого значения (градусы, по умолчанию `-10`)
- t_start** — режим `"time"`: опорное время фазы цикла в формате `HH:MM`
- schedule_len** — режимы `"time"` и `"sun_cycle"`: длина цикла в **секундах** (легаси `schedule_len`, который был в мс). Пусто или `null` — период вычисляется по слотам
- dark_frames** — сколько тёмных кадров снимать на каждую уникальную экспозицию (до и после измерений)
- dead_time** — «мёртвое время», добавляемое к последнему слоту при вычислении периода цикла (с).
Слот со своим `readout` использует его вместо этого значения
- wait_for_enter** — режим `"time"`: ждать нажатия Enter перед стартом. Если терминала нет (`systemd`, `nohup`) — измерения начинаются сразу, независимо от этого флага. В режиме `sun_cycle` не используется: старт задаёт Солнце
- schedule** — список слотов расписания (см. ниже)
- schedule_file** — путь к файлу расписания: старый текстовый `asi-camera` или сконвертированное расписание `imagerd_rt` в JSON. Если задан, он заменяет собой `schedule`
- site_id**, **device_id** — идентификаторы станции и прибора. Попадают в имя файла и в заголовки `SiteID / DeviceID`, по которым их читает программа обработки
- legacy_version** — строка в заголовке `Version` (по умолчанию `"3.0"`, как писал `imagerd_rt`)
- fw_temp** — температура фильтрового колеса для заголовка `FWTemp`. У этого контроллера датчика нет, поэтому по умолчанию `null` → в шапку пишется `-999.0`
- filters** — таблица колеса: `slot`, `wavelength` (тег вида `5577`, `OH__`), `description`. Используется в имени файла и в заголовках `FilterWavelength / FilterDescription`

Подсекция **camera**:

- backend** — `"picam"` (реальный SDK) или `"sim"` (симулятор без оборудования)

- readout_speed** — частота АЦП, МГц (``P.AdcSpeed``)
- binning** — биннинг N×N по всему кадру (частичный ROI камера здесь не использует)
- gain** — аналоговое усиление АЦП: ``1`` Low, ``2`` Medium, ``3`` High
- frame_timeout_ms** — запас времени на считывание сверх самой экспозиции
- readout_control_mode**, **shutter_timing_mode**, **output_signal**, **kinetics_window_height** — параметры PICAM, повторяют настройку легаси-демона
- demo**, **demo_model** — подключить демо-камеру PICAM вместо реальной

Подсекция **cooling**:

- enabled** — использовать охлаждение
- target_temp** — уставка температуры сенсора, °C
- tolerance** — допуск, при котором температура считается достигнутой, °C
- wait_on_start**/**wait_timeout** — ждать выхода на уставку перед измерениями и сколько (с)
- warm_on_exit**/**warm_temp** / **warm_timeout** — отогревать сенсор перед выключением

Подсекция **filter_wheel**:

- port** — последовательный порт контроллера (``sim`` — симулятор)
- baudrate** — скорость порта (по умолчанию ``9600``)
- move_timeout** — предельное время поворота колеса (с). Реальный поворот занимает около секунды; если подтверждения позиции нет, кадр **не** помечается этим фильтром, а в лог идёт предупреждение

Подсекция **location** — **lat**, **lon**, **elevation**: координаты станции.

Записываются в шапку FITS и используются для расчёта высоты Солнца.

Управление интенсивностью

Две необязательные функции, обе по умолчанию выключены. Пока обе выключены, драйвер работает ровно так, как описано выше.

Подсекция **preflight** — предварительная стадия (только **sun_cycle**).

Обычно между «ещё светло» и **sun_max_angle** не снимается ничего: слотовые экспозиции в сумерках дают насыщение. Включённая предварительная стадия занимает этот промежуток. Она стартует по первой, более высокой уставке угла Солнца, идёт по **тому же** основному расписанию — те же слоты, дельты, фильтры, биннинг, усиление, тот же якорь и та же фаза цикла, — но экспозицию каждого слота подбирает сама, удерживая заданную среднюю интенсивность кадра в 16-битных отсчётах АЦП. Когда Солнце доходит до **sun_max_angle**, автоматика выключается и дальше идёт обычный основной цикл, без изменений.

Порядок вечера при включённой стадии: пред-дарки (теперь рассчитаны так, чтобы закончиться к **первой** уставке) → затвор открывается → предварительная стадия → основной цикл. На рассвете сессия завершается там же, где и всегда: при подъёме Солнца выше **sun_max_angle**. Предварительная стадия — вечерняя, утром она не повторяется.

Ключ	По умолчанию	Описание
<code>`enabled`</code>	<code>`false`</code>	включает стадию
<code>`sun_start_angle`</code>	<code>`-6.0`</code>	первая уставка, градусы. Должна быть выше <code>`sun_</code>
<code>`target_mean`</code>	<code>`20000.0`</code>	удерживаемое среднее по кадру, АЦП 0–65535

`tolerance`	`0.15`	мёртвая зона в долях цели: внутри неё экспозиция не
`min_exposure`	`0.05`	нижний предел подбираемой экспозиции, с
`max_exposure`	`null`	верхний предел; `null` = не длиннее экспозиции самого
`max_step`	`4.0`	наибольший множитель изменения экспозиции за одно
`bias`	`0.0`	пьедестал, АЦП; берётся из дарков, этот ключ — запас
`binning`	`null`	биннинг, общий для всех слотов стадии; `null` — кажды

Экспозиция ведётся **по каждому слоту отдельно**: разные фильтры смотрят на небо, различающееся по яркости на порядки. Первый заход на слот начинается с `min_exposure` и растёт: один пересвеченный кадр на старте — как раз то, чего функция должна избежать.

Кадры стадии помечаются в шапке FITS как `OBSMODE = sun_cycle_auto` и, кроме того, суффиксом `_pf` в имени файла — так предварительные кадры отделяются от основных без чтения заголовков.

Подсекция ``overexposure`` — защита от пересветки (цикловые режимы).

Если средняя интенсивность на слоте превысила порог, то в следующий заход на **этот же** слот один кадр слотовой экспозиции `E` разбивается на `N` более коротких подкадров, снимаемых подряд. Мёртвое время на сохранение файла учитывается: бюджет слота — это один кадр плюс одно сохранение, `E + dt`, и в него должны уложиться `N` подкадров вместе с их `N` сохранениями:

$$N \cdot (e + dt) = E + dt - \text{margin}$$

$$e = (E + dt - \text{margin}) / N - dt$$

То есть подкадр чуть короче наивного `E / N` — лишние сохранения надо откуда-то взять. Для расписания станции (`E = 55` с, `readout = 5` с, слоты через 60 с) это даёт 25 с при `N = 2`, 15 с при `N = 3` и 10 с при `N = 4`.

Если одного разбиения мало, `N` растёт по одному за заход — 2, 3, 4, — пока интенсивность не опустится ниже порога либо пока не упрётся в предел. Пределов два: `max_splits` из конфига и арифметический,

$$N \leq (E + dt - \text{margin}) / (e_{\text{min}} + dt)$$

где `e_min` — не меньше `min_frame_gap` за вычетом мёртвого времени. Обратно слот возвращается по прогнозу: измеренная интенсивность линейно пересчитывается на более длинную экспозицию, и как только прогноз для целого кадра оказывается ниже `threshold · release`, слот снова снимает один кадр по программе.

Промежуточные шаги вниз тоже разрешены.

Ключ	По умолчанию	Описание
`enabled`	`false`	включает защиту
`threshold`	`55000.0`	среднее по кадру, выше которого слот дробится, АЦП
`release`	`0.85`	гистерезис возврата: прогноз должен быть ниже `thresh
`max_splits`	`4`	наибольшее число подкадров, не меньше 2
`min_exposure`	`0.05`	наименьшая экспозиция подкадра, с
`margin`	`0.5`	запас, вычитаемый из бюджета слота, с
`min_frame_gap`	`1.0`	наименьший интервал между подкадрами, с; ниже 1 с
`bias`	`0.0`	пьедестал, АЦП; берётся из дарков, этот ключ — запас

`min_frame_gap` не случайно ограничен снизу: имя файла в архиве имеет разрешение в одну секунду, и два подкадра, снятых ближе, претендовали бы на одно имя. Если

мёртвое время слота съедает весь бюджет, разбиение невозможно — слот снимается целым кадром, и об этом один раз пишется в лог.

Про пьедестал: среднее по кадру — это `bias` + сигнал, и с экспозицией растёт только сигнал. Обе функции считают именно по сигналу над пьедесталом, иначе пересчёт «вдвое короче — вдвое темнее» ошибается, причём в опасную сторону. Пьедестал берётся из пред-дарков, которые и так снимаются на каждую уникальную экспозицию; ключ `bias` нужен только когда дарков нет (`dark_frames` = 0 или поздний старт).

Каждый кадр получает в шапке FITS ключ `SKYMEAN` — измеренную среднюю интенсивность в АЦП; подкадры разбиения дополнительно получают `SPLITNUM` (на сколько частей поделён слот) и `SPLITIDX` (номер подкадра, с единицы). Имя файла остаётся прежнего легаси-формата и всегда несёт ту экспозицию, которой кадр действительно снят.

Слоты расписания ASI

Формат слота зависит от режима.

Режим ``sun`` — слоты повторяются каждую минуту, пока Солнце ниже `sun_max_angle`:

```
{ "filter": 1, "exposure": 55, "seconds": [0, 30] }
```

Поле	Описание
<code>`filter`</code>	позиция фильтра, 1–6
<code>`exposure`</code>	экспозиция, секунды
<code>`seconds`</code>	секунды внутри минуты, когда делается кадр

Режимы ``time`` и ``sun_cycle`` — один «общий цикл» повторяется; слоты в обоих режимах одинаковые, различается только то, откуда берётся начало цикла:

```
{ "delta": 100, "filter": 3, "exposure": 25, "binning": 1, "gain": 3, "readout": 5 }
```

Поле	Описание
<code>`delta`</code>	смещение кадра от начала цикла, секунды (легаси <code>`start_time`</code> , было в мс)
<code>`filter`</code>	позиция фильтра, 1–6 (легаси <code>`filter_num`</code>)
<code>`exposure`</code>	экспозиция, секунды (легаси <code>`exposure_len`</code> , было в мс)
<code>`binning`</code>	биннинг для этого кадра (по умолчанию — <code>`camera.binning`</code>)
<code>`gain`</code>	аналоговое усиление 1–3 (легаси <code>`ccd_gain`</code>); пусто — берётся <code>`camera.gain`</code>
<code>`readout`</code>	время считывания и поворота колеса после кадра, секунды (легаси <code>`prep_time`</code>)

Период цикла берётся из `schedule_len`, а если он не задан — вычисляется как последняя `delta` + её экспозиция + её `readout` (при отсутствии `readout` — `dead_time`). Для расписания Тори это `1428 + 7 + 5 = 1440` с, ровно как в старом `schedule_len=1440000`.

Разница между режимами:

- ``time``** — фаза отсчитывается от ``t_start`` по часам, а не от момента запуска программы: если запустить измерения посреди ночи, кадры продолжают попадать в те же секунды, что и при запуске вовремя.
- ``sun_cycle``** — до достижения ``sun_max_angle`` **не снимается ничего**. Программа рассчитывает момент пересечения угла, отступает назад на время съёмки тёмновых и ждёт; тёмновые заканчиваются ровно к открытию окна. Затем цикл привязывается к первой целой минуте окна и крутится до восхода — когда высота Солнца снова

поднимется выше `sun_max_angle`, сессия завершается и снимаются закрывающие тёмновые.

Это поведение легаси-программы `imagerd_rt`.

Необязательная предварительная стадия (`preflight`, см. «Управление интенсивностью») сдвигает открытие окна на первую, более высокую уставку: сумерки между двумя уставками снимаются по тому же расписанию, но с автоподбором экспозиции. Завершение сессии на рассвете не меняется.

Позиции фильтров

№	Фильтр
1	557.7 нм
2	630.0 нм
3	широкополосный ОН
4	840.0 нм
5	846.5 нм
6	857.0 нм

Файлы расписания

`asi.schedule_file` читается вместо `asi.schedule`. Поддерживаются два формата.

Сконвертированное расписание `imagerd_rt` (JSON). В репозитории лежит готовое `schedules/asi_tory_1440.json` — это `schedule.conf_22-11-2022_04-48-36` станции Тори, переведённый в секунды: 36 слотов, цикл 1440 с. Чтобы его включить:

```
"asi": { "schedule_file": "schedules/asi_tory_1440.json" }
```

Соответствие ключей старого `schedule.conf` новым:

<code>imagerd_rt</code> (мс)	<code>every-camera</code> (с)	где
<code>`zenith_angle_start`</code>	<code>`sun_max_angle`</code>	глобально
<code>`schedule_len`</code>	<code>`schedule_len`</code>	глобально
<code>`device ID`</code>	<code>`device_id`</code>	глобально
<code>`start_time`</code>	<code>`delta`</code>	слот
<code>`filter_num`</code>	<code>`filter`</code>	слот
<code>`exposure_len`</code>	<code>`exposure`</code>	слот
<code>`binning`</code>	<code>`binning`</code>	слот
<code>`ccd_gain`</code>	<code>`gain`</code>	слот
<code>`prep_time`</code>	<code>`readout`</code>	слот

Глобальные ключи из JSON-файла (`mode`, `sun_max_angle`, `schedule_len`, `t_start`, `dark_frames`, `dead_time`, `site_id`, `device_id`) перекрывают одноимённые в `config.json` — расписание переезжает целиком, а не по частям. Остальные ключи файла игнорируются. Файл может быть и просто списком слотов.

Старый текстовый файл `asi-camera` (строки с `#` — комментарии):

```
# режим sun:  filter,exposure(s),seconds  - секунды через двоеточие
1,55,0:30
2,55,15

# циклические режимы: delta;filter;exposure(s);binning;gain;readout(s)
100;3;25;1
1428;3;7;4;3;5
```

В циклической строке обязательны только первые три поля: `binning` без него берётся из камеры, а `gain` и `readout` остаются незадаанными — слот тогда работает с общим для камеры

усилением и с общим для расписания `dead_time`, как и любой старый файл из четырёх полей. Поле можно и просто оставить пустым: `0;3;25;1;;5` задаёт `readout`, не задавая `gain`.

Имена файлов и дерево каталогов ASI

Кадры раскладываются по дате в UTC, как это делала `imagerd_rt`:

```
<output_dir>/2026/07/28/20260728_203000_TORY_ASI0_5577_055000ms.fits
<output_dir>/2026/07/28/20260728_202000_TORY_ASI0_OH__007000ms_DARK.fits
<output_dir>/2026/07/28/20260728_193000_TORY_ASI0_5577_000500ms_pf.fits
<output_dir>/seqno.txt          – сквозной счётчик кадров (заголовок SEQNO)
```

Имя файла: `ГГГГММДД_ччммсс_<site_id>_<device_id>_<тег фильтра>_<экспозиция в мс, 6 цифр>ms[_DARK][_pf].fits`.

Ночь разрезается по полуночи UTC — так же, как в старом архиве. Если позиция колеса неизвестна (например, у первых тёмновых, снятых на home-позиции), вместо тега фильтра пишется `none`. Суффикс `_pf` получают кадры предварительной стадии `preflight` — подбор экспозиции в сумерках виден по имени файла, без чтения шапки.

Счётчик `seqno.txt` переживает перезапуски. При переезде со старой программы можно просто скопировать её `aux/seqno.txt` в `output_dir` — нумерация продолжится.

Шапка FITS

Пишутся два набора заголовков. Современный набор `every-camera`:

DATE-OBS (UTC, начало экспозиции), DATE-LOC (местное время), EXPTIME, BINNING, READSPD, FILTER, CCD-TEMP, SETTEMP, GAIN, IMAGETYP (LIGHT/DARK), OBSMODE (sun/time/sun_cycle/dark), INSTRUME, VENDOR, CAMSN, CAMVER, DRVVER, DCAMVER, SITELAT, SITELON, SITEELEV.

Плюс — все 16 полей, которые `imagerd_rt` клала в каждый кадр, ровно под теми же именами (программа обработки читает именно их). Сама `imagerd_rt` сохраняла PNG и писала их в `tEXt`-записи; здесь они стали заголовками FITS:

Имя	Значение
<code>`Binning`</code>	биннинг (совпадает с <code>`BINNING`</code>)
<code>`BitDepth`</code>	разрядность сенсора
<code>`CCDGain`</code>	аналоговое усиление 1–3
<code>`CCDTemp`</code>	температура ПЗС, °C, округлена до сотых
<code>`Exposure`</code>	**строка** вида <code>""55000.00 ms""</code> — миллисекунды
<code>`ReadoutSpeed`</code>	**строка** <code>""2 MHz""</code> или <code>""100 KHz""</code>
<code>`SEQNO`</code>	номер кадра из <code>`seqno.txt`</code>
<code>`SiteID`</code>	<code>`asi.site_id`</code>
<code>`DeviceID`</code>	<code>`asi.device_id`</code>
<code>`Latitude`</code>	широта, округлена до сотых
<code>`Longitude`</code>	долгота, округлена до сотых
<code>`FilterWavelength`</code>	тег фильтра (<code>`5577`</code> , <code>`OH__`</code> , ...)
<code>`FilterPosition`</code>	позиция колеса
<code>`FilterDescription`</code>	описание фильтра
<code>`FWTemp`</code>	температура колеса; датчика нет → <code>`-999.0`</code>
<code>`Version`</code>	<code>`asi.legacy_version`</code> , по умолчанию <code>""3.0""</code>

Имена длиннее 8 символов записываются как HIERARCH-карты, поэтому написание сохраняется буква в букву; `astropy` читает их по тому же имени (`header["FilterWavelength"]`).

Japan (`japan`)

Всенебный имиджер Hamamatsu: камера через DCAM-API плюс то же фильтровое колесо Animatics SmartMotor, что и у ASI. Программа измерений перенесена из отдельной программы japan-camera — она старше и проще программы ASI: два режима расписания вместо трёх, без управления охлаждением и без авто-экспозиции.

Чем именно japan отличается от asi — в разделе

[«Что общего у asi и japan»](#что-общего-у-asi-и-japan).

Расписание живёт прямо в config.json; schedule_file нужен только станции, которая уже ведёт старый schedule.txt от japan-camera.

Основные ключи

- **instance_name** — имя экземпляра; пусто → генерируется автоматически (`JAPAN\_<узел>`)
- **output_dir** — каталог для кадров. Кадры пишутся ****плоско****, без дерева дат, поэтому обычная схема — каталог на ночь (/data/japan/20260729/)
- **mode** — режим расписания: `sun` или `time`. Режим `sun\_cycle` здесь нет, это режим камеры ASI; если он окажется в конфиге, драйвер скажет об этом и возьмёт sun
- **sun_max_angle** — режим `sun`: измерения идут, пока высота Солнца $\leq$ этого значения (градусы, отрицательные — Солнце под горизонтом)
- **t_start** — режим `time`: опорное время фазы общего цикла, `HH:MM` или `HH:MM:SS`
- **schedule_len** — режим `time`: период одного цикла в секундах. `null` или `0` — период выводится из слотов (дельта последнего слота + его экспозиция + dead_time). Файл расписания может задать период сам — см. «Период цикла в самом расписании» ниже; это и есть предпочтительное место, конфиг тогда трогать не нужно
- **dark_frames** — сколько тёмных кадров снимать на каждую уникальную экспозицию
- **dead_time** — время между циклами, зарезервированное на смену фильтра и биннинга (с). Входит в период общего цикла, когда тот выводится из слотов
- **wait_for_enter** — режим `time`: ждать нажатия Enter перед стартом. Без терминала (systemd, nohup) игнорируется — измерения стартуют сразу. На фокусировку это больше не влияет: focus_app.py получает живые кадры и до нажатия Enter
- **schedule_file** — путь к старому `schedule.txt` (или конвертированному `.json`). Если задан, заменяет собой список schedule
- **schedule** — список слотов (см. ниже)

Перезапуск после полуночи. t_start — это время суток, оно наступает раз в 24 часа, и драйвер берёт **ближайшее** из них: если сегодняшнее дальше чем на 12 часов вперёд, значит нужное было вчера. Иначе станция, поднявшаяся в 00:30 при t_start = 22:00, считала бы опорную точку за 21.5 часа впереди — то есть решила бы, что стартовала рано, отсняла бы стартовые тёмные кадры, которые положено пропустить, и при периоде, не делящем сутки нацело, сдвинула бы фазу всех слотов за ночь.

Период цикла в самом расписании

Расписание и длина его цикла — одно целое: период, который живёт только в

config.json, переживает то расписание, для которого был посчитан. Поэтому файл расписания может назвать его сам — строкой ключ = значение в любом месте файла:

```
period = 1440
# delta(s);filter;exposure(s);binning
0;1;55;1
100;3;25;1
```

period — это то же самое, что schedule_len; в .json-расписании ключ называется schedule_len и лежит рядом со slots. Значение из файла старше значения из config.json.

Так же задаются и остальные общие настройки, которые эта камера понимает:

mode, sun_max_angle, t_start, dark_frames, dead_time. site_id и

device_id относятся к архиву ASI и здесь игнорируются. Неизвестный ключ не ломает файл — о нём сообщается, слоты читаются дальше.

Период проверяется по слотам. Период назван в одном месте, а описан в другом — слотами, — и при запуске программа сравнивает одно с другим. Разойтись они могут дорого. Двенадцатиминутный цикл, записанный как period = 1440 (а 12 минут — это 720 секунд), оставляет вторую половину каждого цикла вообще без слотов; фаза считается от t_start, поэтому ближайшим слотом для запуска, случившегося меньше чем за такой период до t_start, оказывается сам t_start — камера стоит до назначенного часа, и по ней не видно, почему. Теперь об этом говорится строкой при старте, и в консоли, и в GUI:

```
! japan: period = 1440 s, but the slots close the cycle at 720 s
(Δ660 + 55 s exposure + 5 s dead time). The last 720 s of every cycle
hold no slot, though its slots are never more than 60 s apart –
one of the two numbers is not what was meant.
```

Ругани на ровном месте не будет: пауза в конце цикла — обычное дело, и говорится только тогда, когда пустой хвост длиннее любого промежутка между слотами внутри цикла, то есть места хватило бы ещё на один слот, а слота нет. Расписание из одного слота на цикл не проверяется вовсе — сравнивать не с чем. Обратный случай — слоты не влезают в заявленный период — сообщается всегда.

Период при этом не исправляется: какое из двух чисел лишнее, знает оператор.

`camera`

- **backend** — `dcam` (настоящий SDK) или `sim` (симулятор, без оборудования)
- **readout_speed** — скорость считывания DCAM `READOUTSPEED`: `1` — медленная (минимум шума), 2 — быстрая. Это перечисление, третьего значения нет
- **binning** — биннинг пикселей, `1`..`8`
- **frame_timeout_ms** — одно ожидание готовности кадра в DCAM. Драйвер ждёт несколько таких окон сверх самой экспозиции и только потом бросает кадр — так зависший wait_capevent_frameready не подвешивает весь воркер

Секции cooling нет: камера сообщает температуру сенсора и не имеет уставки, которую можно было бы задать.

`filter\_wheel` — те же ключи, что у ASI (колесо одно и то же)

- **port** — последовательный порт контроллера (`/dev/ttyUSB0`, `COM1`), либо

sim для симулятора

- **baudrate** — скорость порта (обычно `9600`)
- **move_timeout** — потолок ожидания подтверждения позиции (с). Это защита от застрявшего колеса, а не задержка: обычный переход занимает около секунды

`location` — широта, долгота, высота площадки; нужны для режима sun и пишутся в шапку каждого кадра.

Слоты расписания

Режим sun — кадр снимается на заданных секундах каждой минуты:

```
{"filter": 3, "exposure": 30, "seconds": [0, 30]}
```

Режим time — общий цикл, привязанный по фазе к t_start:

```
{"delta": 100, "filter": 3, "exposure": 25, "binning": 1}
```

delta — смещение от начала цикла (с). Период цикла вычисляется сам:

последняя delta + её экспозиция + dead_time. Для примера
100;3;25;1 / 130;5;25;2 при dead_time = 5 период равен 160 с.

Ключ gain в слоте — это настройка камеры ASI. У Hamamatsu аналогового усиления нет, поэтому такой слот принимается, но драйвер сообщает, что gain проигнорирован.

Старый `schedule.txt`

Формат japan-camera читается как есть. Режим sun — через запятую, фильтр, экспозиция, секунды:

```
3,30,0:30  
5,60,0
```

Режим time — через точку с запятой, delta;фильтр;экспозиция;биннинг:

```
100;3;25;1  
130;5;25;2
```

Строки, начинающиеся с #, и пустые строки пропускаются.

Имена файлов

Кадры пишутся плоско в output_dir:

```
20260729T140530_3.fits      — измерение, фильтр 3  
20260729T140530_3_bg.fits  — тёмный кадр (bg = background)
```

Метка времени — **UTC**. В japan-camera она была локальной; см.

[«Отличия от программы japan-camera»](#отличия-от-программы-japan-camera).

Позиция колеса 0 означает «дома» либо «неизвестно» — так же, как писала старая программа.

Шапка FITS

18 карт, те же, что у ASI, за вычетом того, чего у этой камеры нет (GAIN, SETTEMP, SKYMEAN, SPLITNUM/SPLITIDX) и без блока старых карт imagerd_rt:

Карта	Значение
-------	----------

`DATE-OBS`	начало экспозиции, **UTC**
`DATE-LOC`	то же местное время станции
`EXPTIME`	экспозиция, с
`BINNING`	биннинг
`READSPD`	настройка `READOUTSPEED` — целое `1` или `2`, не мегагерцы
`FILTER`	позиция колеса (`0` — дома либо неизвестно)
`CCD-TEMP`	температура сенсора; нет показания → `-999.0`
`IMAGETYP`	`LIGHT` или `DARK`
`OBSMODE`	`sun`, `time` или `dark`
`INSTRUME`, `VENDOR`, `CAMSN`, `CAMVER`	модель, производитель, серийный номер, прошивка
`DRVVER`	версия драйвера DCAM
`DCAMVER`	версия DCAM API
`SITELAT`, `SITE Lon`, `SITEELEV`	координаты и высота площадки

MQTT (`mqtt`)

- **enabled** — включить/выключить MQTT публикацию
- **host** — адрес брокера (по умолчанию `broker.hivemq.com`)
- **port** — порт (`1883` стандартный, `8883` с TLS)
- **user** — имя пользователя (опционально)
- **password** — пароль (хранится открытым текстом в config.json)
- **prefix** — префикс топиков (по умолчанию `every\_camera`)
- **tls** — использовать TLS-соединение

Структура топиков:

```
{prefix}/{instance_name}/status  - статус камеры (retain=true)
{prefix}/{instance_name}/frame   - последний кадр (JPEG, base64)
{prefix}/{instance_name}/cmd/#   - команды (get_frame, capture_frame)
```

При штатной остановке камера очищает retained-статус, а на случай аварийного обрыва регистрируется MQTT last will со статусом `offline`. Поэтому исчезнувшая камера сразу пропадает из монитора, а не висит в нём как «running» бесконечно.

LAN-сервер кадров (`server`)

- **enabled** — включить HTTP-сервер кадров на машине с камерой
- **bind** — адрес прослушивания (`0.0.0.0` — вся локальная сеть, `127.0.0.1` — только эта машина)
- **port** — *предпочтительный* порт HTTP (по умолчанию `8765`)
- **port_search** — сколько портов выше `port` перебрать, если он занят (по умолчанию 20, 0 — использовать только port). Так несколько экземпляров работают с одним и тем же config.json; в файл ничего не записывается, порт выбирается заново при каждом запуске
- **discovery** — отвечать на запросы поиска камер (multicast `239.255.42.99` и широковещательные UDP)
- **discovery_port** — порт обнаружения (по умолчанию `45455`)
- **max_list** — максимальное число кадров в одном ответе на листинг архива
- **focus_ttl** — сколько секунд камера остаётся в режиме фокусировки после последнего запроса от focus_app.py (по умолчанию 60)

> **Доступ не защищён паролем.** Любой в локальной сети может смотреть кадры и,

> если камера это поддерживает, менять параметры съёмки. Используйте сервер только

> в доверенной сети; чтобы закрыть доступ, поставьте "bind": "127.0.0.1" или
> "enabled": false.

Общие

- node_name** — понятное имя этой машины (например, `Tory-1`). Показывается вместо IP-адреса в viewer_app.py, focus_app.py, discovery.py и в мониторе. Пусто — используется имя хоста. Подробнее: «Имя узла в сети».
- status_dir** — директория для JSON-файлов статуса (по умолчанию `~/every\_camera/status`)

Консоль измерений

В консольном режиме программа рисует **один живой экран**: он перерисовывается на месте, а не прокручивается. Экран одинаков для всех камер — меняется только блок camera, который наполняет драйвер.

```
=====
EVERY CAMERA — ASI @ Tory-1 (ASI_42)
=====
Time:                21:04:33
Status:              running (cycle 12, Δ100 s)
Phase:               measuring
Schedule:            cycle 160 s from 20:00:00 · iteration 12
Next frame:          21:05:00 (in 27.0 s)
Capturing:           LIGHT filter=3  12.4 s / 25 s
Frames captured:      128 light / 6 dark      errors 0
Last file:            20260727T210400_3.fits
Output dir:           /data/asi (812.5 GB free)
---- camera -----
Exposure / binning:   25 s · 2x2
Gain:                 3
Filter:                3
Sensor temperature:   -59.83 °C  setpoint -60.0 °C (locked)
---- network -----
LAN server:           http://192.168.1.42:8765/  name: Tory-1
Live view / focus:    active
MQTT:                 broker.hivemq.com — connected
-----
21:04:30 [INFO ] Saved 20260727T210400_3.fits
21:04:05 [WARN ] Filter wheel did not confirm position 5
=====
Ctrl+C — finish the frame, shoot the closing darks, stop
```

Что показывается:

Строка	Значение
`Time`	часы, тикают независимо от съёмки
`Status` / `Phase`	состояние воркера и текущая фаза (ожидание, тёмные кадры, измерения)
`Schedule`	режим расписания: окно по Солнцу или период цикла
`Next frame`	время следующего кадра и обратный отсчёт
`Capturing`	что снимается прямо сейчас, сколько секунд прошло из экспозиции
`Frames captured`	счётчики кадров и ошибок за сеанс
`Last file`	имя последнего записанного файла
`Output dir`	директория архива и свободное место на диске
блок `camera`	параметры камеры: экспозиция, биннинг, gain, фильтр, температура
блок `network`	адрес LAN-сервера и имя узла, состояние живого просмотра, MQTT
нижняя панель	последние строки лога

Экран не «замерзает» во время экспозиции. Отрисовка идёт в отдельном потоке и читает только словарь состояния; поток отрисовки никогда не обращается к камере, а рабочий поток кладёт в состояние значения (например, температуру) тогда, когда их безопасно читать. Поэтому во время 25-секундного кадра часы и счётчик Capturing продолжают идти.

Отладочный вывод. Драйверы больше не печатают ничего напрямую: все сообщения идут в нижнюю панель и в файл лога. Даже если сторонняя библиотека (gphoto2, stypes-обёртка) что-то напечатает, эта строка перехватывается и попадает в панель, а не поверх экрана.

Файл лога

Полный лог всегда пишется в `~/every_camera/logs/{камера}_{экземпляр}.log` (ротация по 2 МБ, один архивный файл `.log.1`) — независимо от режима отображения.

Плоский режим (`--verbose`)

```
python main.py --type asi --verbose
```

Живой экран заменяется обычными строками ЧЧ:ММ:СС [УРОВЕНЬ] сообщение. То же происходит **автоматически**, если вывод не является терминалом (systemd, nohup, конвейер) — иначе журнал сервиса заполнился бы управляющими escape-последовательностями.

Имя узла в сети

При сканировании локальной сети наблюдатель раньше видел только IP-адреса. Ключ `node_name` в `config.json` задаёт понятное имя машины:

```
{ "node_name": "Tory-1" }
```

Имя описывает **машину**, а не камеру (на одном узле может работать несколько камер) и передаётся:

- в ответе на UDP-обнаружение (`discovery.py`),
- в `/api/info` и `/api/status` LAN-сервера,
- в статусах MQTT и в файлах статуса — монитор показывает его в колонке «Extra Info» как `@Tory-1`.

Где видно:

```
$ python discovery.py --timeout 2
Tory-1          ASI_42          asi          http://192.168.1.42:8801
```

В `viewer_app.py` и `focus_app.py` камеры в списке подписаны

Tory-1 – ASI_42 (asi) – 192.168.1.42:8801, список сортируется по имени узла.

Если ключ не задан, используется имя хоста — поле никогда не бывает пустым.

Задать имя можно в мастере настройки: `setup_app.py` → вкладка **General** →

«Node name», или в консольном мастере при настройке LAN-сервера.

Несколько экземпляров с одним config.json

Сколько угодно копий every-camera работают в одной сети и на одной машине с **одинаковыми** настройками сервера — придумывать новые порты и имена вручную не нужно.

Имя экземпляра. По умолчанию это {камера}_{node_name} (раньше был последний октет IP — он совпадал у машин .5 из разных подсетей). Если такое имя уже занято живым процессом на этой машине, добавляется порядковый номер:

```
[WARN] Instance name 'ASI_Tory-1' is already used by another process on this machine — running as 'ASI_Tory-1-2'.
```

Имя резервируется через flock в ~/.every_camera/instances/, поэтому упавший процесс освобождает своё имя автоматически, а следующий запуск получает его обратно без суффикса. От имени зависят топики MQTT, файл лога и preview_{instance}.png — так что уникальное имя разводит их все сразу.

Порт LAN-сервера. server.port — теперь *предпочтительный*, а не единственный. Если он занят, сервер поднимается на следующем свободном в пределах server.port_search (по умолчанию 20 портов):

```
[INFO] Port 8765 is busy — this instance took 8766. config.json is unchanged; the port is chosen per run.
```

В конфиг ничего не записывается: выбор существует только на время запуска, поэтому config.json остаётся одинаковым на всех узлах. "port_search": 0 возвращает прежнее строгое поведение.

Обнаружение. Зонд рассылается и в multicast-группу 239.255.42.99:45455, и широковещательно, как раньше. Это принципиально: широковещательную датаграмму ядро отдаёт ровно одному сокету, поэтому второй экземпляр на машине был запущен, но невиден. Multicast ядро копирует всем, кто вступил в группу. Broadcast оставлен для коммутаторов без IGMP-querier и для узлов со старой версией.

```
$ python discovery.py --timeout 2
Tory-1      ASI_Tory-1      asi      http://192.168.1.42:8765
Tory-1      ASI_Tory-1-2    asi      http://192.168.1.42:8766
```

> **Обновление существующей установки.** Имена экземпляров изменятся, а значит изменятся и топики MQTT. Retained-статусы под старыми именами останутся у брокера — их нужно один раз почистить (в мониторе они выглядят как «зависшие»).

Тёмные кадры и фазы измерений (ASI и Japan)

Обе камеры ведут сеанс одинаково — эта схема пришла из программы japan-camera и досталась ASI по наследству. Ниже описан сеанс ASI; у Japan он тот же за вычетом того, чего у неё нет: усиления, предварительной стадии preflight, управления интенсивностью и режима sun_cycle.

Сеанс состоит из трёх фаз:

1. **Предварительные тёмные кадры.** dark_frames кадров на каждую уникальную

экспозицию с закрытым затвором; усиление и биннинг берутся у первого слота с этой экспозицией, чтобы тёмновой был вычитаем из своих светлых. В режимах sun и sun_cycle они запускаются так, чтобы закончиться ровно к открытию окна измерений: программа заранее считает момент пересечения Солнцем угла sun_max_angle и вычитает оценку длительности серии. До этого момента не происходит вообще ничего — ни одного кадра.

При включённой предварительной стадии (preflight) окно открывается раньше, и тёмные отсчитываются от первой уставки sun_start_angle, а не от sun_max_angle — иначе они шли бы уже поверх измерений.

В режиме time они снимаются сразу после старта — но только если старт не опоздал относительно t_start (иначе они пропускаются, о чём говорит строка ! note на экране).

Средние по этим кадрам служат пьедесталом для управления интенсивностью (см. «Управление интенсивностью»).

- 2. **Измерения.** Кадры по расписанию (см. «Слоты расписания ASI»). При включённой предварительной стадии эта фаза делится на две: автоматическая съёмка сумерек до sun_max_angle, затем основной цикл.
- 3. **Финальные тёмные кадры.** Снимаются при завершении — в том числе по Ctrl+C, — но только если за сеанс был снят хотя бы один рабочий кадр. Остановка до первого кадра (ожидание сигнала на старт, ещё не начавшиеся сумерки, стадия предварительных тёмновых) финальной серии не порождает: калибровать нечего.

Ctrl+C = «доснять текущий кадр и записать финальные тёмные». Повторный Ctrl+C завершает программу немедленно, без финальной серии.

Что общего у asi и japan

Камера Hamamatsu (japan) и всенебный имиджер PIXIS (asi) — это две программы измерений с общим происхождением: программа ASI была написана на основе программы Hamamatsu и с тех пор ушла вперёд. Часть кода у них не просто похожа, а совпадает — например, фильтровое колесо у них одно и то же, вплоть до команд на проводе. Такой код лежит в cameras/common/ в одном экземпляре:

Модуль	Что это
`filterwheel.py`, `filterwheel_sim.py`	контроллер Animatics SmartMotor: `GOSUB5` — домой, `g=<n>` — переход, `G` — тёмных
`schedule.py`	слоты расписания, тайминг общего цикла, оценка длительности серии
`sun.py`, `timeutil.py`	высота Солнца и одно определение UTC для имён файлов, шапок и расчёта C
`cfgparse.py`	чтение значений из `config.json`, которое никогда не бросает исключение
`fits.py`	18 общих карт шапки FITS и сама запись файла

cameras/common/schedule.py намеренно содержит **объединение** словарей обеих камер: режим sun_cycle и gain в слоте относятся только к ASI. Что из этого принимает конкретная камера, решает её собственный config.py — поэтому конфиг, скопированный от ASI, не срабатывает наполовину молча, а вызывает понятное сообщение.

В cameras/asi/ остались однострочные модули-реэкспорты под старыми именами, чтобы существующие импорты (from cameras.asi import schedule) продолжали

работать. В cameras/japan/ таких нет — пакет новый и импортирует из cameras/common/ напрямую.

Чем `japan` отличается от `asi`

	`asi`	`japan`
Камера	Princeton PIXIS через PICAM	Hamamatsu через DCAM-API
Режимы расписания	`sun`, `time`, `sun_cycle`	`sun`, `time`
Охлаждение	уставка, ожидание, отопев на выходе	нет: только показание температуры сенсора
Усиление	`gain` 1..3	нет
Авто-экспозиция, разбиение слота	есть	нет
Скорость считывания	МГц (`2.0`, `0.1`)	настройка `1` или `2`
Имена кадров	дерево `YYYY/MM/DD` и имена imagerd_rt	плоско, `YYYYmddTHHMMSS_<фильтр>[_bg]].fits`
Шапка FITS	18 общих карт + карты PIXIS + 16 старых карт i14g	18 общих карт
Счётчик `SEQNO`	есть (`seqno.txt`)	нет

Отличия от программы japan-camera

Логика перенесена как есть, но три вещи изменены сознательно — все три касаются данных, поэтому о них лучше знать заранее.

1. `DATE-OBS` и метка времени в имени файла теперь UTC. В japan-camera DATE-OBS содержал **местное** время под комментарием «UTC observation start time», то есть был подписан неверно. Теперь это настоящий UTC, как требует стандарт FITS и как уже сделано у ASI, а местное время сохраняется в новой карте DATE-LOC — чтобы связь с ночным журналом наблюдателя не потерялась. Метка в имени файла тоже UTC. На станции в UTC+8 имена смещаются относительно старого архива на восемь часов; это стоит один раз отметить в журнале.

2. Высота Солнца считается по правильному времени. japan-camera передавала astropу наивное местное время, а astropу читает такое время как UTC — то есть угол Солнца вычислялся для момента, сдвинутого на часовой пояс станции (для UTC+8 — на восемь часов). Ошибка исправлена. ****Следствие: момент открытия окна режима sun изменится**** на любой машине, часы которой идут не по UTC. Это стоит проверить до первой ночи.

3. Съёмка кадра не может зависнуть навсегда. В japan-camera ожидание готовности кадра было бесконечным циклом; при настоящем сбое камеры поток измерений вис, остановка не отработывала, и systemd в итоге убивал процесс посреди экспозиции. Теперь ожидание ограничено: несколько окон frame_timeout_ms поверх самой экспозиции, после чего кадр считается потерянным и цикл идёт дальше.

Кроме того, исправлены две вещи, которые правильнее назвать дефектами, чем отличиями:

- **Серия тёмных кадров в режиме `time` больше не обрывается.** Тёмные кадры идут подряд без синхронизации по секундам, а имя файла различает только секунды — поэтому при экспозиции короче секунды и dark_frames > 1 второй кадр получал то же имя, и запись падала с ошибкой, теряя весь остаток серии. Теперь имя сдвигается на секунду вперёд, пока не найдётся свободное (DATE-OBS при этом хранит настоящий момент начала экспозиции).

- **Неподтверждённый переход колеса не портит имя кадра.**Общее колесо сообщает «позиция неизвестна», если переход не подтвердился; в шапку и в имя файла в этом случае пишется 0, как писала старая программа, а не None.

Не перенесён «оконный» режим расписания (Entry.start / Entry.end и функции вокруг него): в japan-camera он был недостижим — разбор schedule.txt никогда не заполнял эти поля, и обращение к нему упало бы на сравнении с None.

Файл расписания (Canon и Infra)

Формат: одна строка = один интервал измерений.

```
# Measurement Schedule
# Format: %Y-%m-%d %H:%M:%S - %Y-%m-%d %H:%M:%S

2026-03-08 12:00:00 - 2026-03-08 23:59:00
2026-03-09 08:00:00 - 2026-03-09 18:00:00
```

- Строки с # — комментарии
- Камера снимает только когда текущее время попадает в один из интервалов
- Вне интервалов программа ждёт (статус waiting)

Расписание может покрывать сколько угодно ночей вперёд: цикл съёмки рассчитан на то, чтобы жить дольше отдельного интервала, а не завершаться вместе с ним. У камеры Infra сбой одного прохода (отвалившийся кадр, ошибка USB) больше не убивает поток целиком — ошибка пишется в журнал, цикл выдерживает полсекунды и продолжает работу, поэтому одна неудача посреди ночи не отменяет всё оставшееся расписание.

> **Sentry** расписание задаётся иначе — через slots в config.json. Файл schedule.conf для imagerd_rt генерируется автоматически.

Режим настройки (setup mode)

Отсутствующее, пустое или испорченное расписание — не ошибка. Раньше программа в этом случае завершалась с exit(1), и только что установленную камеру нельзя было даже навести: focus_app.py берёт живые кадры у работающего драйвера.

Теперь камера поднимается в **режиме настройки**:

- камера открывается, LAN-сервер кадров работает, focus_app.py подключается;
- плановые съёмки не выполняются, в архив ничего не пишется;
- для ASI не снимаются ни стартовые, ни закрывающие тёмные кадры, но затвор открывается: наводить резкость через закрытый затвор нечем, а открывает его обычно только измерительная программа — после стартовых тёмных кадров;
- статус — setup (виден в мониторе, в /api/status и в MQTT), а в /api/info выставлен флаг setup_mode, по которому focus_app.py рисует метку **SETUP**.

В консоли это выглядит так:

```
[WARN] Setup mode: schedule file not found: /home/alex/data/schedule.txt.
[INFO] No scheduled captures will be taken. Connect focus_app.py to adjust focus
and camera parameters; add a schedule and restart to begin measuring.
[INFO] Focus this camera with: python focus_app.py --host 192.168.1.42 --port 8765
```

Поводы для входа в режим: путь к расписанию не задан, файл не найден, файл пуст или состоит из комментариев, все строки некорректны, файл нечитаем (в том числе двоичный — раньше это давало traceback). Для ASI — пустой `asi.schedule`; для Sentry — пустой `sentry.slots` и отсутствующий `schedule.conf`.

Режим можно включить принудительно, даже когда расписание заполнено:

```
python main.py --type asi --setup
```

Когда расписание появится, достаточно перезапустить программу — она начнёт измерения обычным образом. В GUI пустая таблица расписания больше не блокирует кнопку Start, а стартует ту же вкладку в режиме настройки.

В режим настройки камера входит и по другому поводу — когда оператор подключается к ней через `focus_app.py`, чтобы навести резкость. Это описано в разделе [«Что происходит с измерениями во время настройки»](#что-происходит-с-измерениями-во-время-настройки).

Шкала интенсивности: 0–65535

Все камеры отдают кадры в **одном диапазоне 0–65535**, как 16-битный АЦП. Сенсоры при этом разные: CSDU-429 (SPTT) и Tanho SWIR оцифровывают 12 бит, Canon отдаёт 8-битный JPEG, PIXIS и Hamamatsu — настоящие 16 бит. Приведение делает драйвер сразу после захвата, поэтому статистика, гистограммы, доля насыщенных пикселей, автоэкспозиция и отображение везде означают одно и то же.

Отображение точное: младшие биты дополняются старшими (12 бит: $(v \ll 4) \mid (v \gg 8)$), поэтому нулю соответствует нуль, полной шкале сенсора — ровно 65535, а исходный отсчёт восстанавливается сдвигом вправо. Простой сдвиг $\ll 4$ оставил бы насыщенный пиксель на 65520 — на четыре отсчёта ниже полной шкалы, и ни одна проверка пересвета его бы не увидела.

Каждый кадр несёт свою шкалу в FITS-заголовке:

```
ADCFULL = 65535 / [ADU] intensity full scale
```

Кадры, снятые **до** этого изменения, ключа не содержат и считаются 12-битными: при просмотре они приводятся к 0–65535 на лету, поэтому старый и новый архив выглядят одинаково и сравнимы по числам. Файлы на диске при этом не переписываются.

> **Внешним программам обработки.** Значения в новых файлах отличаются от старых > примерно в 16 раз. Признак — наличие ключа ADCFULL; при его отсутствии файл > следует читать как 12-битный.

Canon и sentry — исключения по записи: JPEG остаётся 8-битным, а FITS от `imagerd_rt` пишет сам демон. Но статистика, которую показывает `viewer_app.py`, приводится к общей шкале и для них.

Режим предпросмотра (preview)

```
# Infra: непрерывная запись preview_Infra.fits/.tiff на максимальной частоте
```

```
python main.py --type infra --preview
```

```
# Canon: то же для preview_Canon.jpg
```

```
python main.py --type cannon --preview

# Sentry: копирование image.fits → preview_Sentry.fits при каждом новом кадре
python main.py --type sentry --preview

# ASI: непрерывная запись preview_ASI_42.fits на максимальной частоте
python main.py --type asi --preview

# Japan: то же для preview_JAPAN_42.fits
python main.py --type japan --preview
```

Опрос камеры ASI и Japan (`--probe`)

```
python main.py --type asi --probe
python main.py --type japan --probe
```

Открывает камеру, печатает её модель, серийный номер, размер сенсора и — главное — какие значения она принимает для каждого параметра, который выставляет драйвер: уставку охлаждения, скорость АЦП, усиление, режимы затвора и считывания, выдержку. В камеру при этом ничего не записывается, поэтому команда работает и тогда, когда запуск падает на «Invalid Parameter Value»: диапазоны в её выводе показывают, какое значение подойдёт вместо забракованного.

Для japan команда отвечает на тот же вопрос — «видит ли DCAM камеру и какие значения она примет» — и печатает таблицу по EXPOSURETIME, BINNING, READOUTSPEED и SENSORTEMPERATURE. После этого она ****прогоняет фильтровое колесо****: возвращает его домой, переводит на позиции 1, 5 и 3 и печатает сырые ответы контроллера с временем. Это перенесённая диагностика wheel_probe.py из japan-camera — так проверяют протокол GOSUB4 → FILT:<n> и измеряют реальное время перехода. В камеру по-прежнему ничего не пишется, но колесо физически двигается, поэтому среди ночных измерений команду запускать не стоит.

При backend = "sim" или port = "sim" соответствующая половина пропускается с пояснением.

Ограничения каждой модели свои, поэтому значение из config.json, которое камера не принимает, драйвер подтягивает к ближайшему допустимому и пишет об этом предупреждение — вместо того чтобы уронить ночь наблюдений:

```
[WARN ] Sensor setpoint (°C): this camera does not accept -60 (allowed: -55 ... 25, step 0.1) - using -55 instead
```

Так обрабатываются измеряемые величины: уставка, скорость АЦП, высота окна Kinetics. А вот режимы (затвора, считывания, выходного сигнала, усиления) никогда не подменяются «ближайшим»: соседний номер — это другой режим, а не почти тот же самый, и молча снятые с закрытым затвором кадры хуже остановки. Для них запуск прекращается с тем же понятным сообщением, где перечислены допустимые значения. Выдержка тоже не подрезается — укороченная экспозиция незаметно испортила бы измерение.

В графическом режиме вкладка **Infra Camera** имеет встроенный **Live Preview** для настройки резкости и параметров камеры в реальном времени:

1. Нажмите **Connect** для подключения к камере
2. Нажмите **Live Preview** для запуска потока видео

- 3. Настройте **Exposure**, **Gain** и **ROI** — изменения применяются немедленно
- 4. Нажмите **Stop Preview** когда настройка завершена

Live Preview и режим измерений по расписанию работают независимо.

Мониторинг

```
python monitor_app.py
```

monitor_app.py показывает каждую камеру плиткой: что это, на какой машине работает, чем занята сейчас и что пошло не так.

```
python monitor_app.py           # найти камеры в локальной сети
python monitor_app.py --host 192.168.1.5 # добавить камеру, которую не видно обнаружением
python monitor_app.py --interval 5      # период опроса статусов, секунд
python monitor_app.py --mqtt          # сразу подключить брокер из config.json
```

Источников два, и они неравноправны:

- **Локальная сеть — основной.**Камеры находятся тем же UDP-обнаружением, что и в viewer_app.py / focus_app.py, а плитка заполняется из GET /api/status самой камеры. Настраивать нечего, показания берутся из первых рук и обновляются раз в несколько секунд. Кнопка **Search LAN** повторяет поиск вручную, флажок **Keep searching** — раз в 20 секунд, чтобы запущенная позже камера появилась сама.
- **MQTT — вторичный,** для камер на другой площадке, куда нет маршрута до их HTTP-порта. Панель внизу окна (свёрнута по умолчанию) подключает брокер, и их ретейненные статусы становятся такими же плитками. Камера, видимая обоими путями, показывается по локальной сети: это прямое показание, а не пересказ через брокер.

На плитке: статус-бейдж, тип камеры, имя машины и адрес, затем показания, зависящие от типа — фаза и режим расписания, экспозиция и биннинг, положение колеса фильтров и затвора у ASI, температура сенсора с уставкой, число кадров, время последнего кадра, обратный отсчёт до следующего слота, свободное место, число ошибок. Отдельной строкой отмечается режим настройки и то, что кто-то сейчас наводит эту камеру.

Кнопки на плитке ничего не скачивают сами, а передают камеру нужной программе:

Frames... запускает viewer_app.py, **Focus...** — focus_app.py с её адресом.

Для камеры, доступной только по MQTT, они выключены — маршрута до неё нет.

Details показывает статус целиком, как он опубликован.

- > Раньше монитор сам запрашивал кадры по MQTT — это был второй, более слабый
- > просмотрщик внутри монитора. Теперь кадры целиком за viewer_app.py.
- > Старая таблица с вкладками MQTT и Local files никуда не делась: она осталась
- > вкладкой **Monitor** в gui_app.py.

Статусы

Статус	Цвет	Описание
‘RUNNING’	зелёный	Камера активно снимает
‘WAITING’	оранжевый	Ожидание расписания
‘ERROR’	красный	Ошибка при съёмке
‘STOPPED’	серый	Остановлена
‘OFFLINE’	оранжевый	Камера не отвечает (причина написана на плитке)

‘SETUP’	оранжевый	Режим настройки: расписания нет, в архив ничего не п
‘STALE’	оранжевый	Нет обновлений более 30 секунд

Камера, пропавшая из ответов на обнаружение, не исчезает из окна: плитка остаётся с пометкой OFFLINE, потому что «камера, которая была и перестала отвечать» — это ровно то, ради чего монитор и открыт. Одиночный потерянный UDP-ответ ничего не меняет: пропуска считаются, и нужно два подряд.

Во вкладке **Monitor** в gui_app.py (старая таблица) записи процессов, которых больше нет в системе, помечаются как STOPPED и (dead) — раньше упавший процесс висел в списке как работающий.

Просмотрщик кадров (‘viewer_app.py’)

Программа наблюдателя: только просмотр, ничего не снимает и не меняет настроек камеры. Работает на Windows и Linux, драйверы камер не нужны.

```
python viewer_app.py           # найти камеры в локальной сети
python viewer_app.py --host 192.168.1.5  # сразу подключиться к камере
python viewer_app.py --mqtt          # открыть сразу вкладку HiveMQ
```

Вкладка «Local network»

Полный доступ к архиву камеры:

- 1. **Search LAN** — широковещательный запрос, в списке появляются все найденные камеры. Если обнаружение выключено или заблокировано, введите хост:порт вручную.
- 2. Выпадающий список дат и список кадров — можно открыть **любой** снятый кадр, хоть месячной давности.
- 3. **Latest frame** — последний кадр; **Follow live** обновляет его каждые 3 секунды.
- 4. **Contrast** — растяжка гистограммы для 16-битных кадров: min–max, перцентильная (0.5–99.5 %, лучше видно слабые детали рядом с яркими пикселями) или без растяжки.
- 5. **Save original...** — скачать исходный файл (FITS/TIFF/PNG/JPEG) без преобразований.

Список кадров пополняется сам. Раз в пять секунд просмотрщик спрашивает камеру, не вырос ли архив, и перечитывает его только если вырос — вопрос стоит одну строку листинга, а не весь архив. Раскрытые ночи и циклы, положение прокрутки и выбранный кадр переживают это обновление, так что за деревом можно следить, не трогая его. Кнопка **Reload** осталась для случая, когда каталог камеры изменился как-то иначе — например, файлы удалили вручную.

Архив обходится вглубь. ASI раскладывает кадры по дереву ГГГГ/ММ/ДД, остальные камеры пишут в одну директорию; список кадров и список дат собираются одинаково в обоих случаях, на любой глубине вложенности. Кадр с датой в имени открывается и тогда, когда запрошен просто по имени: сервер сам находит его в подкаталоге соответствующего дня. Выход за пределы output_dir при этом по-прежнему запрещён.

Справа — панель **Frame info** (выключается одноимённым флажком). Это единственное место, где описан кадр, и порядок строк в ней — порядок вопросов: какой это кадр и когда снят; что в нём измерено (размер, тип данных, минимум/максимум/среднее, доля насыщенных пикселей, шкала интенсивности); и затем весь FITS-заголовок таблицей

«ключ — значение» — экспозиция, фильтр, усиление, биннинг, температура ПЗС и её уставка, тип кадра, режим наблюдений, номер в последовательности. Ниже — гистограмма интенсивности по всему диапазону 0–65535 с логарифмической высотой столбцов. Она отвечает на вопрос, который средним и максимумом не решается: кадр «сидит» в пьедестале, размазан по диапазону или упёрся в насыщение.

Раньше под изображением была ещё и подпись в три строки. Она набиралась мелким кеглем и почти целиком повторяла ту же таблицу; единственное, чего в таблице не было, — измеренное по пикселям, и оно теперь стоит в ней третьим блоком. Одно место лучше двух: просмотрщик, называющий экспозицию кадра дважды, вынуждает гадать, какое из двух значений сейчас верное.

Панель работает и в режиме Follow live. FITS-заголовок у живого кадра нет — драйвер публикует пиксели и три-четыре факта о них, — поэтому камера сообщает вместе с кадром путь только что записанного файла, а сервер читает заголовок из него той же функцией, которой описывает архив. Следствие: кадр в режиме Follow live и он же, открытый из архива минутой позже, описаны одинаково. Если файла нет — режим setup ничего не пишет, у Canon нет заголовка, — панель показывает файловые данные, измеренное и то, что камера всё же опубликовала.

Архив читается прямо с диска камеры, поэтому кадры доступны, даже если камера сейчас занята измерениями или её рабочий поток завис.

Период цикла берётся у самой камеры

Номер цикла нигде в кадре не записан — ни в имени файла, ни в шапке FITS, — поэтому просмотрщик восстанавливает период из самих кадров (по тому, через сколько возвращается один и тот же фильтр). Это догадка: ночь, потерявшая слот, или фильтр, снятый в цикле дважды, могут её сдвинуть.

Поэтому камера теперь сообщает своё расписание в `/api/info` — режим, период и опорное время, причём **и в UTC тоже**, так как имена кадров UTC, а `t_start` станции локальный. При подключении к такой камере поля **Period** и опорное время заполняются её значениями и включаются автоматически, и группировка идёт по настоящему периоду, а не по угаданному. Стоит вам поправить эти поля руками — они остаются вашими, и расписание камеры их больше не трогает.

Архив, отданный без камеры (`python -m frame_server --output-dir ...`), тоже может знать свой период — передайте ему `--camera-type japan --config config.json`.

Разностный кадр по фильтру (только japan)

Флажок **Filter difference** (по умолчанию выключен, рядом с `**Group by cycle and filter**`) — при выборе ветки фильтра внутри цикла рядом с основным окном открывается второе, с обработанным кадром:

```
A = первый кадр этого цикла для этого фильтра
B = последний кадр предыдущего цикла для того же фильтра
D = последний кадр этого цикла для того же фильтра

k0 = clamp(1 - (t_A - t_B) / T, 0, 1)
k1 = clamp(1 - (t_D - t_A) / T, 0, 1)
```

$$\text{результат} = A - (B \cdot k_0 + D \cdot k_1) / 2$$

Вес тем ближе к 1, чем меньше промежуток: соседний кадр, снятый в тот же момент, входит целиком, а дальше T не входит вовсе. T задаётся прямо в этом окне (по умолчанию — период цикла), и результат пересчитывается на лету. В окне видны сами k_0 , k_1 , промежутки и имена всех трёх исходных кадров.

Окно **не открывается**, когда считать нечего: у первого цикла ночи нет предыдущего, у самого свежего цикла ещё нет последнего кадра, а если между циклами в архиве пропуск — предыдущим не назначается цикл постарше. Причина в таких случаях пишется в строку состояния. Растяжка берётся из того же **Contrast**, что и у основного окна, — иначе два кадра рядом было бы не сравнить.

Три кадра выбирает сам просмотрщик — из того же дерева, которое вы видите, так что картинка не может разойтись с веткой, которая её запросила. Камера получает три имени и считает арифметику там, где пиксели уже лежат (/api/composite): три японских кадра — это около 24 МБ по сети против ~200 КБ у готового JPEG.

Вкладка «HiveMQ»

Для камер, доступных только через интернет. Здесь запрашивается **только последний кадр** — архив по MQTT не листается, чтобы не гонять мегабайты base64 через публичный брокер.

1. Заполните параметры брокера и нажмите **Connect**.
2. В таблице появятся камеры, публикующие статус.
3. Выберите камеру и нажмите **Request latest frame**.

Настройка резкости (`focus\_app.py`)

Помощник наведения резкости: живой поток кадров, изменяемые параметры съёмки и числовая оценка резкости. **Работает только по локальной сети** — живой поток через публичный брокер потребовал бы неприемлемого трафика.

```
python focus_app.py # найти камеры в локальной сети
python focus_app.py --host 192.168.1.5 # сразу подключиться
```

Порядок работы:

1. **Search LAN** → выбрать камеру → **Start live view**.
2. Крутить фокусер, наблюдая за числом резкости и графиком: при точной фокусировке значение достигает максимума (100 % of best), а зелёная пунктирная линия показывает лучший результат за сеанс.
3. Навести **измерительный квадрат** на нужную звезду (см. ниже) — резкость считается только по нему.
4. Панель **Camera parameters** строится по ответу самой камеры, поэтому у каждой камеры свои поля. Изменить значение → **Apply**.
5. **Stop** возвращает камеру в обычный режим.

Строка под графиком предупреждает о пересвете (`overexposed`), если насыщено больше 1 % пикселей.

Что происходит с измерениями во время настройки

Подключение `focus_app` **приостанавливает расписание** — одинаково у всех камер.

Раньше живые кадры выдавались только в промежутках между экспозициями, у каждой камеры по-своему, и оператор не мог предсказать, что получит: на длинном слоте картинка обновлялась раз в минуту, а изменённый параметр применялся неизвестно когда. Теперь камера на время сеанса входит в режим настройки и отдаёт кадры без ограничений.

- Если камера **сейчас ведёт измерения**, `focus_app` спрашивает подтверждение: «Pause and focus» / «Leave it measuring». Пропущенные за время настройки слоты не доснимаются.
- Если камера **в setup-режиме или вне окна наблюдений** (день, промежуток между интервалами расписания), предупреждения нет — прерывать нечего.
- **Stop**, закрытие окна или потеря связи возвращают камеру к расписанию: удержание снимается вместе с сеансом фокусировки, а при обрыве связи — по тайм-ауту (не дольше 10 минут). Расписание продолжается со следующего подходящего слота.
- Уже начатая экспозиция никогда не прерывается: приостановка означает «не начинать новых кадров».
- Исключение — **sentry**: его расписанием владеет демон `imagerd_rt`, приостановить его отсюда нельзя. Живой поток работает, а `focus_app` сообщает об этом в статусной строке.

Пока удержание действует, в панели «Connection» горит ``measurements paused for focusing`, а в `monitor_app.py`` тайл камеры пишет «Measurements paused while someone focuses this camera» — чтобы остановленная камера не выглядела сломанной.

Измерительный квадрат

Слева показан весь кадр, справа — **отдельная панель с содержимым квадрата**, растянутым на всю её ширину. Наводиться удобно по общему виду, а судить о резкости — по увеличенному фрагменту, и переключаться между ними не нужно.

Управление:

- **клик или перетаскивание** по кадру — навести квадрат на эту точку;
- **Ctrl + колесо** над кадром — изменить сторону квадрата;
- **Side** — сторона в пикселях, любая, не только из списка; ``0`` («whole frame») возвращает измерение по всему кадру;
- **Centre X / Y** — координаты центра в пикселях, если нужно точное значение;
- **Centre** — вернуть квадрат в середину кадра;
- **Zoom main view too** — заменить и левый вид одним квадратом, когда общая композиция уже не важна;
- **Smooth the square preview** — сглаживание увеличенного фрагмента. По умолчанию выключено: сглаживание дорисовывает те самые границы, по которым идёт оценка.

Центр квадрата хранится в долях кадра, а не в пикселях, поэтому наведение не сбивается при смене **Max size** потока. Квадрат у края кадра сдвигается внутрь, а не обрезается — заказанный размер измеряемой области сохраняется. Смена размера или положения сбрасывает график: другой участок неба означает другую шкалу резкости.

Камера	Живой поток	Изменяемые параметры
Canon	Live View (`get_preview`), при отказе — обычный снимок	`iso`, `shutterspeed`, `aperture`, `whitebalance`
SPTT	непрерывный захват между плановыми снимками	`exposure`, `gain`, `binning`, `encoding`
Infra	поток кадров, который камера и так поддерживает	`exposure_us`, `gain`, `roi`
Sentry	кадры демона `imagerd_rt` по мере появления	нет: расписание задаётся в `schedule.conf` и требует п
ASI	кадр снимается в паузе между плановыми, если успевает дождаться фокуса	`exposure`, `binning`, `roi`, `shutter`, `target_temper
Japan	то же, что у ASI	`exposure`, `binning`, `readout_speed`, `filter`, `shutter`

- > У ASI и Japan живой кадр берётся только тогда, когда до следующего планового
- > снимка остаётся больше, чем экспозиция плюс запас. Фокусировка физически не
- > может задержать измерение. Параметры применяются между экспозициями, а
- > следующий слот расписания возвращает свои значения.
- > У SPTT период кадра в этом списке отсутствует намеренно: он следует за
- > экспозицией автоматически, и период, который можно выставить из формы, — это
- > период, который можно выставить короче экспозиции, снова обрезав накопление
- > (см. [«Экспозиция и период кадра»](#экспозиция-и-период-кадра)). Первый кадр
- > после смены параметров отбрасывается, поэтому в фокусировке видно уже новое
- > значение, а не то, что камера успела накопить со старым.
- > **Камера отзывается сразу, в каком бы состоянии ни была.** Раньше в режиме
- > `time c wait_for_enter` воркер ждал Enter в цикле, который публиковал статус
- > и применял параметры, но не снимал живых кадров: `focus_app` подключался,
- > показывал `stream: streaming` — и пустое окно, пока кто-нибудь не подходил к
- > машине и не нажимал Enter. Теперь через один и тот же обработчик проходит
- > **каждое** ожидание драйвера, включая ожидание Enter, и живой кадр приходит
- > за доли секунды. Затвор при этом открывается для сеанса фокусировки сам и
- > закрывается обратно, когда сеанс кончается или начинаются измерения, — без
- > этого кадр был бы просто тёмным.
- >
- > Единственное, что по-прежнему может отказать в живом кадре, — плотное
- > расписание (до следующего слота меньше, чем нужно на экспозицию) и съёмка
- > тёмных кадров, где затвор принадлежит им. Оба случая теперь называют причину:
- > она приходит в `/api/status` полем `focus_note` и видна в `focus_app`.
- > **Затвор ASI.** Затвором управляет контроллер колеса фильтров, а не камера, и
- > измерительная программа держит его закрытым до конца стартовых тёмных кадров.
- > Поэтому в форме параметров есть флажок **Shutter — open**: без него живой
- > кадр в такой момент оказывался бы просто тёмным. Состояние затвора камера
- > сообщает обратно (оно же видно в консоли, строка `Shutter:`), а закрытие
- > затвора во время работы расписания драйвер сопровождает предупреждением —
- > тёмными станут и плановые кадры. Значение принимается и словом (`open`,
- > `closed`), что удобно для команды `cmd/capture_frame` по MQTT.
- > **Колесо фильтров ASI: `home` — это не «неизвестно».** После запуска и связи с
- > контроллером колесо встанёт в положение `home` (0), и монитор с `focus_app`
- > показывают именно `home`. Раньше в этот момент писалось `unknown` — тем же
- > словом, которым обозначается настоящая беда: подтверждения прихода в позицию не
- > было, колесо может быть где угодно, и подписывать такой кадр номером фильтра

> нельзя. Теперь это разные состояния: home — известное место, unknown — только
> несостоявшийся поворот. В именах файлов и заголовке FITS оба по-прежнему 0,
> чтобы не заводить второе написание для уже разобранного архива.
>
> В форме параметров home показывается, но выбрать его нельзя — колесо встаёт
> туда само при запуске, это состояние, а не позиция, куда его посылают (в списке
> он серый). Выбрать можно только реальные фильтры 1..6. Когда положение
> неизвестно, поле остаётся пустым: показывать последний известный фильтр значило
> бы утверждать то, чего никто не знает. В схеме параметров такие значения лежат
> в states, а не в choices — это разница между «камера может *быть* в этом» и
> «этого у неё можно *попросить*».

Значения в форме — это показания, а не то, что было при подключении

Камера меняет свои настройки сама: расписание переключает фильтр и выдержку между кадрами, затвор ASI открывается после стартовых тёмных. Поэтому:

- драйвер публикует текущие значения не один раз при старте, а на той же частоте, что и статус (`worker_common.publish_current_params`);
- `focus_app.py` перечитывает `GET /api/params` раз в несколько секунд, пока идёт живой просмотр, и обновляет поля формы.

Обновление не спорит с оператором: поле, которое он изменил и ещё не отправил, сохраняет введённое значение и помечается звёздочкой (Shutter *), а во всплывающей подсказке написано, что на этот счёт сообщает камера. Пометка гаснет, как только камера сообщает применённое значение — то есть после **Apply** сама собой, — а также по кнопке **Reset fields**.

Отправляется только то, что оператор действительно менял: форма узнаёт об этом от самих виджетов, а не сравнением значений. Иначе поле, о котором камера ничего не сообщает (колесо фильтров, пока положение неизвестно), выглядело бы как правка — и **Apply** заодно отправлял бы команду, которую никто не давал.

Почему это не мешает измерениям

- Плановая съёмка всегда в приоритете: рядом с секундами расписания (`capture_seconds`) кадры для фокусировки не берутся, поэтому снимок не может задержаться.
- Камера снимает в свободном режиме, только пока `focus_app.py` подтверждает, что за ней смотрят. У запроса есть срок (`server.focus_ttl`, по умолчанию 60 с): закрытие окна, обрыв сети или аварийное завершение программы сами возвращают камеру к работе по расписанию.
- Пять ошибок подряд при съёмке кадров фокусировки — режим выключается сам.

HTTP-сервер кадров и обнаружение в сети

На каждой машине с камерой поднимается небольшой HTTP-сервер (`frame_server.py`). Он запускается автоматически вместе с измерениями — и в консольном режиме, и в GUI.

Сервер устроен так, чтобы никогда не мешать измерениям:

- работает в отдельных даемон-потоках; при занятом порте выводит предупреждение и измерения продолжают как ни в чём не бывало;
- обработчики запросов **не обращаются к камере** — живые кадры берутся из потокобезопасной прослойки, архив читается с диска;
- ограничено число одновременных видеопотоков и размер листинга;
- имя файла проверяется на выход за пределы директории архива.

Точки доступа

Запрос	Назначение
`GET /`	Встроенная веб-страница — можно смотреть кадры прямо из браузера
`GET /api/info`	Имя экземпляра, тип камеры, возможности
`GET /api/status`	Текущий статус воркера
`GET /api/dates`	Даты, за которые есть кадры
`GET /api/frames?date=&limit=&offset=`	Листинг архива
`GET /api/composite?first=&previous=&last=&window=&stretch=&max=`	Разностный кадр из трёх названных кадров (japan)
`GET /api/frame?name=&as=jpeg\`	raw&max=&stretch=`
`GET /api/thumb?name=`	Миниатюра
`GET /api/stats?name=`	Статистика и гистограмма кадра (без `name` — живого)
`GET /api/latest.jpg`	Последний живой кадр
`GET /api/live.mjpg?fps=&max=`	MJPEG-поток (используется `focus_app.py`)
`GET /api/params`	Текущие параметры и схема редактируемых полей
`POST /api/params`	Запрос на изменение параметров
`POST /api/focus`	Включить/продлить/выключить режим фокусировки

Просмотр архива без камеры

Сервер можно запустить отдельно, просто над директорией с кадрами — например, на машине, куда снимки скопированы:

```
python -m frame_server --output-dir /path/to/output --port 8765 --node-name "Архив"
```

Чтобы такой архив ещё и раскладывался по циклам, серверу нужно расписание, по которому он снимался, — его берут из конфига:

```
python -m frame_server --output-dir /path/to/output --camera-type japan \  
--config config.json
```

Дальше к нему подключается viewer_app.py или обычный браузер. Ключ --node-name задаёт имя, под которым узел появится в списке при сканировании сети (по умолчанию — имя хоста).

Директорию можно указывать как на плоский архив, так и на дерево ГГГГ/ММ/ДД, которое пишет ASI: сервер обходит вложенные каталоги сам, отдельного ключа для этого не нужно.

Обнаружение камер

```
python discovery.py --timeout 2
```

Рассылает широковещательный UDP-запрос и печатает все ответившие камеры:

Tory-1	ASI_42	asi	http://192.168.1.42:8801
Sura-2	Infra_17	infra	http://192.168.1.17:8765

Первая колонка — имя узла (node_name, см. «Имя узла в сети»), затем имя экземпляра, тип камеры и адрес. Тот же механизм использует кнопка **Search LAN**.
Если сеть блокирует широковещательные пакеты, адрес всегда можно ввести вручную.

Генератор расписания по углу Солнца

Утилита `schedule_generator.py` автоматически рассчитывает время захода/восхода Солнца через заданный угол высоты и создаёт файл расписания для камер Canon и Infra.

Использование

```
# Интерактивный режим (выбор предустановленной точки наблюдения)
python schedule_generator.py

# С параметрами командной строки
python schedule_generator.py --lat 51.81 --lon 103.08 --alt 680 \
    --angle -10 --start 2026-01-01 --days 100 --output schedule.txt

# С предустановленной точкой
python schedule_generator.py --site GPHO_TK --angle -10 --start 2026-01-01 --days 100 -o schedule.txt

# Графический режим
python schedule_generator.py --gui
```

Параметры

Параметр	Описание
`--lat`	Широта (градусы)
`--lon`	Долгота (градусы)
`--alt`	Высота над уровнем моря (метры)
`--angle`	Пороговый угол высоты Солнца (градусы, например `-10`)
`--start`	Начальная дата (`YYYY-MM-DD`)
`--days`	Количество дней для расчёта
`--output` / `-o`	Путь к выходному файлу расписания
`--site`	Предустановленная точка наблюдения
`--gui`	Запустить в графическом режиме

Предустановленные точки наблюдения

Код	Название	Широта	Долгота	Высота
`GPHO_TK`	GPHO Tory	51.8106°	103.0755°	680 м
`KLNG_1`	Kaliningrad	54.6033°	20.2111°	30 м
`MAGNITKA`	Magnitka	55.9305°	48.7449°	95 м
`SURA_ISZF`	SURA ISZF	56.1471°	46.0985°	172 м
`SURA_NNGU`	SURA NNGU	56.1502°	46.1050°	184 м
`SURA`	SURA	56.1437°	46.0990°	173 м
`PEREVOZ`	Perevoz	55.5426°	44.5357°	155 м

Зависимости: `astropy` (входит в требования SPTT-камеры).

Обновление

Утилита `update.py` обновляет программу до последней версии из `git`-репозитория.

Обновляются только скрипты (.py), директории cameras/, firmware/, infra_lib/ и requirements.txt. Конфигурация (config.json, camcfg_*.ini) и расписания (*schedule*.txt) не затрагиваются.

```
# Обновить до последней версии
python update.py

# Посмотреть что изменится (без применения)
python update.py --dry-run

# Перезаписать даже локально изменённые файлы
python update.py --force
```

Запуск через bash-скрипт (run.sh)

Кроме python3 main.py, программу можно запускать через run.sh — обычную bash-обёртку. Всё, что идёт после имени скрипта, передаётся в main.py как есть, так что это прямая замена:

```
./run.sh --type asi
./run.sh --type asi --config /etc/every-camera/asi.json
./run.sh --gui
```

Что скрипт берёт на себя:

- переходит в каталог программы, чтобы относительные пути вели себя предсказуемо;
- выбирает интерпретатор: переменная PYTHON, иначе venv рядом с программой (venv/ или .venv/), иначе python3 из PATH;
- подхватывает env.sh, если он есть, — туда складывается всё машинно-зависимое: пути PICAM SDK, активация conda, выбор интерпретатора. Файл в .gitignore, образец — env.sh.example:

```
cp env.sh.example env.sh # и поправить под свою машину
```

Именно через run.sh запускается и служба systemd, поэтому окружение описывается **один раз** и запуск руками совпадает с запуском службой.

Для ASI в env.sh обычно нужен блок Pleora eBUS / GenICam: драйвер грузит libpicam.so через ctypes (cameras/asi/picam.py), а PICAM для GigE-камер тянет за собой eBUS-рантайм. Эти переменные читает не Python, а нативные библиотеки, поэтому логин-шелл их ставит, а служба не получает ничего — это самая частая причина, почему камера работает руками и не работает под systemd. Готовый блок со значениями станции — в env.sh.example.

Отдельная тонкость — кэш GenICam (GENICAM_CACHE_V2_4). Каталог нужен записываемый, и GenICam не создаёт его сам. Не стоит класть его в общий /tmp/genicam_cache_v2_4: служба работает от root, оператор запускает программу от себя, и кто первый создал каталог — тот им и владеет; второй потом не сможет туда писать, а проявится это как «камера у одного открывается, у другого нет». Путь внутри \$HOME снимает вопрос, а HOME в юните и так выставлен правильно.

Последняя строка run.sh — это exec, и это не косметика. Без него bash остался бы процессом-родителем, SIGTERM от systemctl stop прилетел бы в него, а Python остался бы без корректного завершения — без финальных тёмных и без отогрева сенсора. С exec процесс один, и сигнал приходит туда, где он обрабатывается.

Если хочется автозапуск именно скриптом, без `systemd`, — это возможно (``cron @reboot, rc.local``, автозапуск сессии), но менее удобно: не будет ни перезапуска при падении, ни `systemctl status`, ни журнала, ни корректного таймаута на остановку при выключении ПК. Для станции лучше `systemd` — ниже.

Запуск как службы `systemd`

Программа умеет работать под `systemctl` — в том числе стартовать сама после перезагрузки машины. Обычный запуск руками при этом никуда не девается: `python3 main.py --type asi` работает ровно как раньше, со своим `config.json` из каталога программы. Служба — это дополнительный способ запуска, а не замена.

В отличие от легаси-демона `imagerd_rt`, здесь `Type=simple`: программа не демонизируется, поэтому не нужны ни PID-файл, ни скрипт-обёртка `start|stop`. Процессом владеет сам `systemd`, и `systemctl status` с журналом говорят о нём правду.

Установка

```
systemd/install.sh asi --dry-run --user alex    # посмотреть, что будет записано
sudo systemd/install.sh asi                    # собственно установка
```

Скрипт подставляет в шаблон `systemd/every-camera@.service` пути этой копии программы, кладёт юнит в `/etc/systemd/system/`, копирует `config.json` в `/etc/every-camera/asi.json` (если его там ещё нет) и делает `systemctl enable --now`. После этого камера поднимается при загрузке ПК.

Юнит шаблонный, имя после `@` — тип камеры, поэтому на одной машине можно держать несколько камер одновременно:

```
sudo systemd/install.sh asi
sudo systemd/install.sh japan
sudo systemd/install.sh cannon
systemctl status every-camera@asi
journalctl -u every-camera@asi -f
```

Каждая служба читает свой файл `/etc/every-camera/<тип>.json`. Имена экземпляров разводятся сами (`utils.claim_instance_name`), так что MQTT-топики, файлы логов и превью не пересекаются.

Что важно знать

Остановка не мгновенная, и торопить её нельзя. По `systemctl stop` драйвер досниает текущий кадр, пишет финальные тёмные, а у ASI ещё и подогревает сенсор с рабочей температуры. Только на прогрев отводится до `asi.cooling.warm_timeout` (900 с по умолчанию). Штатные для `systemd` 90 с прибили бы процесс посреди этого, поэтому в юните стоит `TimeoutStopSec=1800`. Если уменьшать — уменьшать осознанно.

Служба работает от `root`, но пишет в домашний каталог оператора. Логи, файлы статуса и блокировки имён экземпляров живут в `$HOME/.every_camera/`. У `root` это

был бы `/root/.every_camera/`, и `monitor_app.py`, запущенный пользователем, камеры бы просто не увидел. Поэтому в юните стоит `Environment=HOME=/home/<user>` — скрипт установки подставляет туда того, кто вызвал `sudo`. Один комплект файлов на оба способа запуска.

Проверьте, какой интерпретатор попал в юнит. По умолчанию берётся первый `python3` из `PATH`, а это часто `anaconda` из домашнего каталога пользователя — для системной службы плохой выбор: каталог принадлежит пользователю, а служба работает от `root` и стартует до того, как что-либо в `/home` гарантированно готово. Лучше указать системный интерпретатор явно:

```
sudo systemd/install.sh asi --python /usr/bin/python3
```

Тогда, разумеется, зависимости (`requirements-asi.txt`) должны быть установлены именно для него — либо укажите путь к `python` из `venv`, созданного под службу.

PICAM SDK не подхватится сам. Переменные окружения, которые обычно ставит логин-шелл, службе не достаются. Прописывать их в юните не нужно — они идут в `env.sh` (см. раздел про `run.sh` выше), который читается и службой, и при запуске руками. После правки: `systemctl restart every-camera@asi`.

Живой экран сам превращается в обычные строки лога, когда вывод не терминал, так что `--verbose` службе не нужен — журнал и так читаемый. Ожидание `Enter` в режиме `time` тоже отключается само.

Удаление

Всё обратимо:

```
sudo systemd/install.sh asi --uninstall
```

Скрипт останавливает службу **штатным путём** (то есть с финальными тёмными и отогревом сенсора), снимает автозапуск и удаляет файл юнита — но только если им не пользуется другая камера, юнит-то шаблонный. Конфиг в `/etc/every-camera/`, архив кадров и логи остаются: терять программу станции из-за одного флага — не то поведение, которого ждут от скрипта удаления. Если нужно снести и их — вручную.

То же самое руками:

```
sudo systemctl disable --now every-camera@asi
sudo rm /etc/systemd/system/every-camera@.service
sudo systemctl daemon-reload
```

Ручной юнит

Если ставить руками, минимум такой (пути — свои):

```
[Unit]
Description=Every Camera – ASI
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=root
WorkingDirectory=/home/alex/every-camera
ExecStart=/usr/bin/python3 /home/alex/every-camera/main.py --type asi \
```



```
--config /etc/every-camera/asi.json
Environment=HOME=/home/alex
Restart=on-failure
RestartSec=30
KillSignal=SIGTERM
TimeoutStopSec=1800

[Install]
WantedBy=multi-user.target
```

--config обязательно абсолютным путём: по умолчанию конфиг ищется рядом со скриптом, а рабочий каталог службы может быть любым.

Остановка

- Консольный режим: Ctrl+C — программа корректно останавливает все потоки
- Под systemd: systemctl stop шлёт SIGTERM, который обрабатывается **так же**, как Ctrl+C — с финальными тёмными и отогревом сенсора. Повторный сигнал (systemd шлёт его по исчерпанию TimeoutStopSec) означает «выйти немедленно, без финальных тёмных» — это лучше, чем быть убитым SIGKILL посреди экспозиции
- GUI: закрыть окно — конфигурация сохраняется автоматически (в тот файл, который был загружен через --config), MQTT отключается
- Sentry: при остановке драйвер посылает сигнал stop_file=10 демону imagerd_rt; демон завершит текущий цикл и остановится
- ASI: первый Ctrl+C (или SIGTERM) — доснять текущий кадр и записать финальные тёмные кадры; повторный — выйти немедленно. При выходе сенсор отогревается, если включён cooling.warm_on_exit
- При остановке снимается retained-статус в MQTT и удаляется файл статуса, поэтому камера сразу исчезает из монитора и просмотрщика
- LAN-сервер кадров останавливается вместе с измерениями: после этого архив по сети недоступен, пока камера не будет запущена снова (или пока не будет запущен `python -m frame_server --output-dir ...` отдельно)

USB-права (Linux)

Для работы с камерами без sudo:

Canon (gphoto2)

```
# Обычно работает из коробки. Если нет — убить gvfs:
sudo pkill gvfsd-gphoto2
```

SPTT (CSDU-429)

```
echo 'SUBSYSTEM=="usb", ATTR{idVendor}=="04b4", MODE="0666"' \
| sudo tee /etc/udev/rules.d/99-sptt.rules
sudo udevadm control --reload-rules
```

Infra (SW1300)

```
echo 'SUBSYSTEM=="usb", ATTR{idVendor}=="aa55", ATTR{idProduct}=="8866", MODE="0666", GROUP="plugdev" \
| sudo tee /etc/udev/rules.d/99-sw1300-camera.rules
sudo udevadm control --reload-rules && sudo udevadm trigger
```

Sentry (imagerd_rt)

Демон `imagerd_rt` обращается к камере напрямую через SDK (Picam + Pleora eBUS). Права настраиваются согласно документации Keo Scientific / Princeton Instruments для конкретной модели. Every-camera запускает `imagerd_rt_start` как пользовательский процесс — для этого достаточно прав на исполнение бинарного файла.

ASI (PIXIS + фильтровое колесо)

Камера PIXIS работает через PICAM SDK — права на USB-устройство настраиваются установщиком SDK от Princeton Instruments. Отдельно нужен доступ к последовательному порту фильтрового колеса:

```
sudo usermod -aG dialout $USER      # затем перелогиниться
ls -l /dev/ttyUSB0                  # проверка: группа должна быть dialout
```

Если библиотека установлена не в стандартный каталог, укажите путь явно:

```
export PICAM_LIB=/opt/PrincetonInstruments/picam/runtime/libpicam.so
```

Japan (DCAM-API + колесо)

Камера работает через `libdcamapi.so` из DCAM-API SDK Hamamatsu; библиотеку ставит установщик SDK, обычно в `/usr/local/lib`. Обёртки Hamamatsu (`dcam.py`, `dcamapi4.py`) уже лежат в `cameras/japan/dcamsdk/vendor/` — отдельно ставить их не нужно. Доступ к последовательному порту колеса — тот же, что у ASI:

```
sudo usermod -aG dialout $USER      # затем перелогиниться
ls -l /dev/ttyUSB0                  # проверка: группа должна быть dialout
```

Если библиотека установлена не в стандартный каталог, укажите путь явно — `LD_LIBRARY_PATH` здесь не поможет, потому что путь задаётся именем файла:

```
export DCAM_LIB=/opt/hamamatsu/dcam/libdcamapi.so
```

Переменная читается и при запуске через `run.sh`, и под `systemd`, если прописать её в `env.sh` (см. `env.sh.example`). При неудачной загрузке драйвер печатает все опробованные пути, так что неверное значение видно сразу.

Тесты

Тесты не требуют ни камеры, ни PICAM SDK, ни DCAM-API SDK — всё, что нужно оборудование, прогоняется на симуляторах:

```
pip install pytest
python -m pytest tests -q
```

Файл	Что проверяет
<code>tests/test_asi_schedule.py</code>	тайминг циклов ASI, фаза относительно <code>`t_start`</code> , оценка тёмных кадров, разбиение
<code>tests/test_asi_exposure.py</code>	авто-экспозиция и арифметика разбиения слота
<code>tests/test_asi_preflight.py</code>	стадия предварительных измерений в сумерках
<code>tests/test_asi_split.py</code>	разбиение по переэкспозиции от начала до конца на симуляторах

`tests/test_asi_sun_cycle.py`	режим `sun_cycle`: порядок событий за ночь
`tests/test_asi_archive.py`	дерево архива ASI и старые карты шапки imagerd_rt
`tests/test_asi_fits_header.py`	шапка FITS камеры ASI карта за картой, в порядке — страховка при правках о
`tests/test_asi_shutter.py`	затвор и позиции колеса
`tests/test_asi_setpoint.py`	уставка охлаждения и её ограничения
`tests/test_asi_picam.py`	соответствие ctypes-биндинга PICAM заголовку `picam.h`: значения параметро
`tests/test_japan_config.py`	чтение секции `japan`, отказ от чужих режимов и значений, старый `schedule.tx
`tests/test_japan_fits.py`	шапка FITS камеры Japan: 18 карт и ничего сверх, UTC в `DATE-OBS`
`tests/test_japan_naming.py`	плоские имена файлов и защита от совпадения имён в одну секунду
`tests/test_japan_driver.py`	режимы `sun` и `time` от начала до конца на симуляторах
`tests/test_japan_sdk.py`	драйвер импортируется без DCAM SDK; локальная правка вендорного файла
`tests/test_japan_params.py`	правка параметров из `focus_app` и отказ от чужих
`tests/test_sptt_exposure.py`	экспозиция SPTT: период кадра выводится из экспозиции и следует за ней, об
`tests/test_sptt_firmware.py`	поиск прошивки SPTT: каталог по умолчанию, `firmware_dir`, отсутствие файло
`tests/test_cannon_exposure.py`	выдержка Canon из `config.json` доходит до камеры, а неподдерживаемая отве
`tests/test_stop_signals.py`	остановка по SIGINT/SIGTERM во всех драйверах, `run.sh`, юнит systemd
`tests/test_console_ui.py`	консоль измерений: отрисовка, плоский режим, перехват чужого вывода, рота
`tests/test_focus_params.py`, `tests/test_param_sync.py`	синхронизация параметров с `focus_app`
`tests/test_monitor_tiles.py`	плитки монитора по типам камер
`tests/test_multi_instance.py`, `tests/test_instance_names.py`	несколько экземпляров с одним `config.json`
`tests/test_setup_mode.py`, `tests/test_focus_roi.py`	режим настройки и измерительный квадрат
`tests/test_live_stats.py`	описание живого кадра: заголовок читается из только что записанного файла,

Генерация PDF документации

```
pip install fpdf2
python generate_pdf.py
# → README.pdf
```